



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Instytut Informatyki

Praca dyplomowa magisterska

POST-KWANTOWE ALGORYTMY PODPISU CYFROWEGO - ANALIZA WYBRANEGO ALGORYTMU

Hubert Mamys, 144257

Promotor

dr hab. inż. Anna Grocholewska-Czuryło

POZNAŃ 2024

Spis treści

1	Wstęp	2
2	Przegląd literatury	5
3	Dlaczego RSA może być wkrótce podatny na złamanie	9
3.1	Wykorzystanie RSA w podpisie cyfrowym	9
3.2	Algorytm Shora	10
3.3	Konkurs NIST	12
3.4	Kryptografia oparta na kratach	13
3.4.1	Problemy i schematy w kryptografii opartej na kratach	15
3.4.2	Wykorzystanie w kryptografii	19
4	CRYSTALS-Dilithium	20
4.1	Uprozczone kroki	20
4.2	Rozszerzona interpretacja	24
4.3	Algorytmy pomocnicze	27
4.4	Przykładowy przebieg algorytmu CRYSTALS-Dilithium	31
4.4.1	Algorytm generowania klucza	31
4.4.2	Algorytm podpisu	33
4.4.3	Algorytm weryfikacji podpisu	35
4.5	Porównanie wydajności algorytmów <i>Dilithium</i> , <i>FALCON</i> , <i>SPHINCS+</i>	36
5	Wnioski i perspektywy rozwoju kryptografii postkwantowej	41
	Literatura	43

Streszczenie

W szybko rozwijającej się dziedzinie kryptografii, tradycyjne schematy podpisów cyfrowych, takie jak RSA i ECDSA, stają się coraz bardziej podatne na potencjalne ataki związane z rozwojem komputerami kwantowymi. To zbliżające się zagrożenie spowodowało rozwój kryptografii postkwantowej, której celem jest tworzenie algorytmów kryptograficznych odpornych na takie ataki. Jedną z najbardziej obiecujących gałęzi kryptografii postkwantowej zdaje się być kryptografia oparta na kratkach. Narodowy Instytut Standaryzacji i Technologii (NIST) zorganizował konkurs mający na celu standaryzację postkwantowych algorytmów szyfrowania i podpisu cyfrowego. Jednym z wybranych finalistów konkursu został algorytm *CRYSTALS-Dilithium*. Jest on algorytmem podpisu cyfrowego opartym na problemie Learning With Errors (LWE) oraz jego modyfikacji Module Learning With Errors (MLWE). Cechuje się wysokim poziomem bezpieczeństwa oraz efektywnością obliczeniową, co czyni go jednym z czołowych kandydatów do standaryzacji w kontekście zabezpieczeń postkwantowych. Celem pracy było zbadanie i omówienie rosnącego zagrożenia dla tradycyjnych schematów podpisów cyfrowych przez rozwój komputerów kwantowych oraz analiza podejść kryptografii postkwantowej, które mogą zapewnić odporność na tego rodzaju ataki. Praca skupiła się na szczególnym algorytmie postkwantowym, jakim jest *CRYSTALS-Dilithium*. W ramach pracy dokonano szczegółowej analizy algorytmu, omawiając jego teoretyczne podstawy oraz wydajność w odniesieniu do pozostałych finalistów konkursu.

Abstract

In the rapidly evolving field of cryptography, traditional digital signature schemes such as RSA and ECDSA are becoming increasingly vulnerable to potential attacks associated with the development of quantum computers. This upcoming threat has led to the development of post-quantum cryptography, aimed at creating cryptographic algorithms resistant to such attacks. One of the most promising branches of post-quantum cryptography appears to be lattice-based cryptography. The National Institute of Standards and Technology (NIST) organized a competition aimed at standardizing post-quantum encryption and digital signature algorithms. One of the selected finalists of the competition was the *CRYSTALS-Dilithium* algorithm. It is a digital signature algorithm based on the Learning With Errors (LWE) problem and its modification Module Learning With Errors (MLWE). It is characterized by high security levels and computational efficiency, making it one of the top candidates for standardization in the context of post-quantum security. The aim of the study was to investigate and discuss the growing threat to traditional digital signature schemes posed by the development of quantum computers, as well as to analyze approaches in post-quantum cryptography that could provide resilience against such attacks. The work focused on a specific post-quantum algorithm, *CRYSTALS-Dilithium*. The study conducted a detailed analysis of the algorithm, discussing its theoretical foundations and performance in comparison to other finalists of the competition.

Rozdział 1

Wstęp

Podpis cyfrowy jest zaawansowaną formą elektronicznego podpisu, który umożliwia weryfikację autentyczności i integralności wiadomości, dokumentów oraz danych cyfrowych. W przeciwieństwie do tradycyjnego podpisu odręcznego, podpis cyfrowy opiera się na skomplikowanych technikach kryptograficznych, które zapewniają znacznie wyższy poziom bezpieczeństwa i wiarygodności. Jest to kluczowy element współczesnych systemów zabezpieczeń, umożliwiający bezpieczne przesyłanie informacji w środowisku cyfrowym. Podpis cyfrowy pełni kilka kluczowych i fundamentalnych funkcji znanych w procesie bezpiecznej wymiany wiadomości używając systemów kryptograficznych. Przede wszystkim zapewnia autentyczność, gwarantując, że podpisana wiadomość pochodzi od określonego, najprawdopodobniej potwierdzonego nadawcy. Dzięki temu odbiorca może być pewien, że otrzymana wiadomość nie została sfałszowana. Zapewnia również integralność, co oznacza, że zawartość wiadomości nie została zmieniona po jej podpisaniu. Jest to niezwykle istotne w kontekście transmisji danych, gdzie każda modyfikacja mogłaby mieć poważne konsekwencje. Ostatnią z trzech istotnych funkcji jest niezaprzeczalność, która umożliwia nadawcy wyparcie się swojego podpisu po wysłaniu wiadomości. Wszystkie te cechy przyczyniają się do wysokiego poziomu bezpieczeństwa, chroniąc przed fałszerstwami i nieautoryzowanymi modyfikacjami.

Podpis cyfrowy znajduje szerokie zastosowanie w różnych dziedzinach. W bankowości i finansach jest używany do autoryzacji transakcji finansowych i dokumentów, co pozwala na bezpieczne zarządzanie środkami finansowymi w środowisku online. Administracja publiczna korzysta z podpisów cyfrowych w ramach usług e-urzędu do podpisywania umów, decyzji administracyjnych czy e-faktur, co ułatwia i przyspiesza procesy administracyjne. W handlu elektronicznym podpisy cyfrowe są wykorzystywane w procesach zakupowych, kontraktach i dokumentach sprzedażowych, co zwiększa zaufanie między stronami transakcji. Poczta elektroniczna używa podpisów cyfrowych do weryfikacji tożsamości nadawcy i integralności wiadomości, co pomaga w ochronie przed oszustwami i phishingiem. Ponadto, podpis cyfrowy jest kluczowy w oprogramowaniu do podpisywania kodu, co zapewnia jego integralność i autentyczność źródła. Dzięki temu użytkownicy mogą być pewni, że pobrane oprogramowanie pochodzi od zaufanego źródła i nie zostało

zmodyfikowane.

Według najnowszej wersji standardu FIPS 186-5[2023][1], zatwierdzonego przez amerykański Narodowy Instytut Standaryzacji i Technologii (*National Institute of Standards and Technology, NIST*), algorytm DSA (Digital Signature Algorithm) został wycofany z użytku. Oznacza to, że DSA nie jest już rekomendowany ani wspierany do zastosowań związanych z cyfrowym podpisem, dopuszczone jest jednak wykorzystywanie go do potwierdzania podpisów wykonanych za pomocą tego algorytmu.

FIPS 186-5 (Federal Information Processing Standard) wprowadza zmiany i aktualizacje mające na celu zwiększenie bezpieczeństwa i efektywności kryptograficznych metod podpisu cyfrowego. W najnowszej wersji tego standardu dozwolone pozostają dwa główne algorytmy kryptograficzne:

- RSA (Rivest–Shamir–Adleman) - to jeden z najstarszych i najbardziej rozpowszechnionych algorytmów kryptograficznych. Opiera się na trudności faktoryzacji dużych liczb pierwszych. Jest szeroko stosowany w różnych protokołach bezpieczeństwa, takich jak SSL/TLS, a także do podpisów cyfrowych i szyfrowania danych.
- ECDSA (Elliptic Curve Digital Signature Algorithm) - to algorytm kryptograficzny oparty na krzywych eliptycznych. Jest uważany za bardziej efektywny pod względem bezpieczeństwa i wydajności w porównaniu do tradycyjnych algorytmów takich jak RSA. Dzięki mniejszym kluczom oferuje porównywalny poziom bezpieczeństwa, co czyni go szczególnie przydatnym w środowiskach o ograniczonych zasobach, takich jak urządzenia mobilne i IoT.

Wycofanie algorytmu DSA wynika z jego ograniczeń w zakresie bezpieczeństwa oraz wydajności w porównaniu do nowocześniejszych algorytmów [2]. Przejście na RSA i ECDSA zapewnia wyższy poziom bezpieczeństwa kryptograficznego, co jest kluczowe w dzisiejszych zastosowaniach cyfrowych podpisów, w których integralność i autentyczność informacji są kluczowe.

Motywacją do analizy algorytmu postkwantowego podpisu cyfrowego *CRYSTALS-Dilithium* było rosnące znaczenie kryptografii postkwantowej we współczesnym świecie. Postęp technologiczny, a w szczególności rozwój komputerów kwantowych, stawia przed nami nowe wyzwania w zakresie bezpieczeństwa informacji. Tradycyjne metody kryptograficzne, takie jak RSA, mogą stać się podatne na ataki ze strony zaawansowanych komputerów kwantowych, co wymaga opracowania nowych, odpornych na tego typu zagrożenia algorytmów. Kryptografia postkwantowa oferuje rozwiązania, które są w stanie sprostać tym wymaganiom, zapewniając bezpieczeństwo obecnie wytwarzanych i magazynowanych danych, jak i tych, które jeszcze nie zostały wygenerowane, w nadchodzącej erze postkwantowej.

Celem pracy była szczegółowa analiza algorytmu *CRYSTALS-Dilithium* oraz zrozumienie mechanizmów kryptografii postkwantowej, w szczególności tej opartej na kratkach. Praca miała również na celu zbadanie, dlaczego kryptografia postkwantowa jest niezbędna w kontekście zagrożeń związanych z rozwojem komputerów kwantowych. Omówione zostało także zastosowanie

algorytmu RSA w podpisie cyfrowym oraz zagrożenia wynikające z algorytmu Shora na ten algorytm. Przedstawiono również pokrótce proces standaryzacji najlepszych pod względem bezpieczeństwa i wydajności algorytmów, które spełniają wymogi i poziomy bezpieczeństwa NIST w perspektywie odporności na ataki kwantowe, oraz omówiono kryptografię opartą na kratkach, w tym problemy i schematy w niej występujące oraz jej zastosowanie w kryptografii.

Praca składa się z kilku rozdziałów. W rozdziale pierwszym opisany został ogólny kontekst i motywacje stojące za analizą. Dodatkowo przedstawiono pokrótce stan obecnie wykorzystywanych algorytmów oraz zachodzące w kontekście bezpieczeństwa zmiany. W rozdziale trzecim przeprowadzono przegląd literatury wprowadzającej oraz rozszerzającej tematykę bezpieczeństwa, wydajności oraz wykorzystania algorytmu *CRYSTALS-Dilithium*. Rozdział trzeci omawia, dlaczego algorytm RSA, wykorzystywany w algorytmach podpisu cyfrowego, może wkrótce stać się podatny na złamanie, skupiając się na podatności problemu faktoryzacji liczb wykorzystywanej w tym algorytmie z wykorzystaniem algorytmu Shora. Następnie przedstawiony został koncept kryptografii opartej na kratkach, jako dominującej gałęzi kryptografii postkwantowej wśród finalistów konkursu NIST. Na końcu zdefiniowane i opisane zostały główne problemy i schematy wykorzystywane jako podstawy bezpieczeństwa kryptografii postkwantowej. Rozdział czwarty jest poświęcony szczegółowej analizie algorytmu *CRYSTALS-Dilithium*, w tym podstawowym krokom w formie uproszczonej, ułatwiającym zrozumienie ogólnego konceptu algorytmu. Następnie przeprowadzona została szczegółowa analiza głównych kroków oraz algorytmów pomocniczych. Na koniec zaprezentowany został przykładowy przebieg każdego z kroków algorytmu, w oparciu o przykłady na małych liczbach i w 2 wymiarach, w celu jasnego i przejrzystego przedstawienia przeprowadzanych obliczeń. Na zakończenie, w rozdziale piątym, przedstawiono wnioski i perspektywy rozwoju kryptografii postkwantowej, omawiając możliwe kierunki badań i wdrożeń oraz znaczenie tych technologii dla przyszłości bezpieczeństwa informacji.

Rozdział 2

Przegląd literatury

Celem przeglądu literatury było zebranie, analizowanie i podsumowanie dostępnych materiałów naukowych dotyczących algorytmu postkwantowego podpisu cyfrowego *CRYSTALS-Dilithium*. Algorytm ten zyskał na znaczeniu po 2016 roku, kiedy to wziął udział w konkursie NIST na postkwantowe algorytmy kryptograficzne. W 2022 roku został ogłoszony finalistą, a w 2023 powstały zarysy standardów NIST FIPS 203, 204, 205. Z uwagi na relatywnie krótki czas od jego wprowadzenia i ciągły brak gotowego standardu, liczba dostępnych materiałów naukowych nie jest jeszcze bardzo obszerna.

W ramach przeglądu literatury sformułowano pięć kluczowych pytań badawczych:

PB1: Jakie są granice bezpieczeństwa algorytmu Crystals-Dilithium w świetle postępów w dziedzinie algorytmów kwantowych?

To pytanie koncentruje się na analizie, czy i w jakim stopniu algorytm *CRYSTALS-Dilithium* jest odporny na ataki z wykorzystaniem komputerów kwantowych. Chodzi o zrozumienie, czy istnieją potwierdzone dowody na jego odporność na tego typu zagrożenia.

PB2: Jakie są obliczeniowe i komunikacyjne koszty generowania i weryfikacji podpisów cyfrowych za pomocą algorytmu *CRYSTALS-Dilithium*?

Badanie to ma na celu określenie, jakie zasoby są potrzebne do skutecznego generowania i weryfikacji podpisów cyfrowych. Dotyczy to zarówno kosztów obliczeniowych, jak i wymagań dotyczących przepustowości sieci.

PB3: Jak algorytm *CRYSTALS-Dilithium* może być zoptymalizowany pod kątem wydajności na różnych platformach sprzętowych?

To pytanie skupia się na identyfikacji możliwości optymalizacji algorytmu w zależności od specyficznych platform sprzętowych, na których jest implementowany. Analiza obejmuje zarówno optymalizacje programowe, jak i sprzętowe.

PB4: Jak algorytm *CRYSTALS-Dilithium* radzi sobie pod względem bezpieczeństwa i wydajności w porównaniu do innych postkwantowych algorytmów podpisu

cyfrowego?

W ramach tego pytania przeprowadzono porównanie algorytmu *CRYSTALS-Dilithium* z innymi finalistami konkursu NIST oraz innymi popularnymi algorytmami postkwantowymi, analizując ich mocne i słabe strony.

PB5: Jakie są wyzwania związane z wdrażaniem algorytmu *CRYSTALS-Dilithium* w realnych systemach i urządzeniach?

Pytanie to ma na celu zidentyfikowanie praktycznych problemów i barier, które mogą wystąpić podczas implementacji algorytmu w rzeczywistych systemach oraz urządzeniach.

W Tabeli 2.1. przedstawiono kryteria włączające podczas wyszukiwania materiałów, natomiast Tabela 2.2. przedstawia kryteria wykluczające dla wyszukanych prac i materiałów.

TABELA 2.1. Kryteria włączające do przeglądu literatury

Kryteria włączające
Data publikacji: po 2015 roku
Typ publikacji: artykuł, materiały konferencyjne, standardy, dokumentacje techniczne
Słowa kluczowe: optymalizacja, analiza wydajnościowa, analiza bezpieczeństwa, analizy porównawcze
Metodyka analizy: symulacje komputerowe, implementacje sprzętowe/programowe
Język pracy: język angielski

TABELA 2.2. Kryteria wykluczające do przeglądu literatury

Kryteria wykluczające
Data publikacji: przed 2016 rokiem
Literatura opisująca ogólną koncepcję kryptografii postkwantowej
Literatura poświęcona kryptografii opartej na kratkach

Etap poszukiwania materiałów obejmował systematyczne przeszukiwanie baz danych takich jak DBLP, ACM, oraz przegląd materiałów organizacji IACR i NIST. Wyszukiwania były przeprowadzane z użyciem odpowiednich słów kluczowych, takich jak *CRYSTALS-Dilithium*, *post-quantum digital signatures*, *cryptographic resistance to quantum computers*, *performance analysis of CRYSTALS-Dilithium*, oraz *security analysis of Crystals-Dilithium*.

Proces wyszukiwania literatury został podzielany na dwa główne etapy:

- Przeszukiwanie elektronicznych baz artykułów naukowych przy użyciu określonych fraz i słów kluczowych.

- Dokładna analiza znalezionej literatury pod względem podobieństwa do zadanych pytań badawczych i wybór tej spełniającej kryteria

Analiza wyników poszukiwań pozwoliła na wyselekcjonowanie najbardziej adekwatnych materiałów. Przegląd literatury umożliwił ocenę aktualnego stanu wiedzy na temat algorytmu Crystals-Dilithium:

Evaluating the security of CRYSTALS-Dilithium in the quantum random oracle model

Badanie [3] skupia się na analizie bezpieczeństwa algorytmu Crystals-Dilithium w kontekście ataków kwantowych. Identyfikowane są trzy kluczowe problemy obliczeniowe: MSIS, SIS oraz SelfTargetMSIS. Autorzy wprowadzają nowy dowód trudności dla problemu SelfTargetMSIS, wykorzystując redukcję z MLWE w modelu kwantowego losowego wyroczni (QROM). Propozycja nowych zestawów parametrów dla Dilithium, które zapewniają bezpieczeństwo przy $q = 1 \bmod 2n$, ma na celu zmniejszenie liczby operacji, co przekłada się na poprawę wydajności obliczeniowej. Artykuł podkreśla potrzebę dalszych badań w bardziej realistycznych modelach kwantowych i wskazuje na pozytywne perspektywy algorytmu Crystals-Dilithium jako potencjalnego kandydata na odporny na komputery kwantowe algorytm podpisu cyfrowego.

Security Comparisons and Performance Analyses of Post-Quantum Signature Algorithms

W artykule [4] przedstawione zostało szczegółowe porównanie czterech algorytmów odpornych na komputery kwantowe: CRYSTALS-Dilithium, Rainbow, SPHINCS+ i SIKE. Dilithium wyróżnia się jako najszybszy z analizowanych algorytmów, choć wymaga większych kluczy i podpisów. Rainbow oferuje mniejsze klucze i podpisy, jednak kosztem wydajności. SPHINCS+ charakteryzuje się dużą szybkością generowania i weryfikacji podpisów, przy zachowaniu dużych rozmiarów kluczy. SIKE jest kompromisem między bezpieczeństwem, wydajnością a rozmiarem kluczy. Artykuł wskazuje, że algorytmy post-kwantowe są już konkurencyjne lub nawet lepsze od tradycyjnych metod opartych na krzywych eliptycznych, co sugeruje możliwość migracji bez znacznego pogorszenia wydajności.

LWE with Side Information: Attacks and Concrete Security Estimation?

Autorzy pracy przedstawiają [5] opracowane narzędzia do kryptoanalizy algorytmów opartych na kratkach, ze szczególnym uwzględnieniem ataków side-channel przy użyciu narzędzia Sage 9.0. Omówiono także atak skip-addition na algorytm Dilithium, który wykorzystuje błędy dodawania do wyodrębnienia fragmentu tajnego klucza i generowania fałszywych podpisów. Artykuł podkreśla potrzebę wzmocnienia odporności algorytmów na tego typu ataki.

A Novel Power Analysis Attack against CRYSTALS-Dilithium Implementation

Zaprezentowane w pracy [6] zostały dwie nowe techniki typu power analysis (PA) na implementację NTT w algorytmie Dilithium. Autorzy demonstrują, że ich zoptymalizowane podejście jest znacznie szybsze od istniejących metod, co stanowi poważne wyzwanie dla bezpieczeństwa algorytmu. Ataki te wymagają fizycznego dostępu do urządzenia, na którym działa Dilithium, co podkreśla potrzebę stosowania odpowiednich środków ochronnych.

A High-Performance Hardware Architecture for the CRYSTALS-Dilithium Signature Scheme

Artykuł [7] opisuje zaawansowaną architekturę sprzętową opartą na układzie FPGA, dedykowaną kluczowym operacjom kryptograficznym w algorytmie Dilithium. Układ FPGA oferuje znaczne przyspieszenie w porównaniu do implementacji programowych, przy niskim poborze mocy. Rozwiązanie to pokazuje, że algorytm Dilithium może być efektywnie realizowany w sprzęcie, co stanowi istotny postęp w jego praktycznym wdrożeniu.

Każdy z tych artykułów dostarcza głębszego zrozumienia specyficznych aspektów badawczych i technologicznych związanych z algorytmem *CRYSTALS-Dilithium* oraz podkreśla aktualne wyzwania i możliwości w dziedzinie kryptografii odpornych na komputery kwantowe.

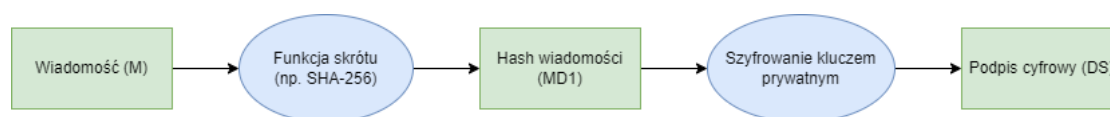
Rozdział 3

Dlaczego RSA może być wkrótce podatny na złamanie

W obliczu rosnącego zagrożenia, jakie niesie rozwój komputerów kwantowych, Narodowy Instytut Standaryzacji i Technologii (NIST) w Stanach Zjednoczonych ogłosił w 2016 roku konkurs na opracowanie nowych standardów kryptograficznych odpornych na ataki kwantowe [8]. Konkurs ten, znany jako NIST Post-Quantum Cryptography (PQC) Standardization, miał na celu wybór algorytmów szyfrowania i podpisu cyfrowego, które będą w stanie zapewnić bezpieczeństwo w erze postkwantowej.

3.1 Wykorzystanie RSA w podpisie cyfrowym

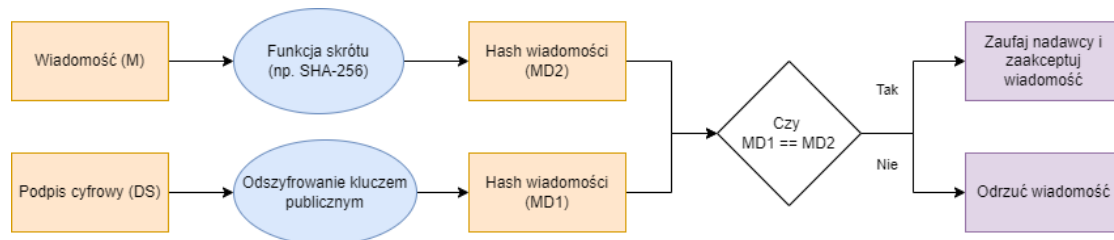
Podczas procesu podpisywania z wykorzystaniem algorytmu RSA przedstawionego na Rysunku 3.1., nadawca tworzy skrót MD1 wiadomości M za pomocą funkcji skrótu, na przykład SHA-256. Funkcja skrótu przekształca wiadomość dowolnej długości w krótką, unikalną wartość o stałej długości. Kolejnym krokiem jest szyfrowanie tego skrótu za pomocą prywatnego klucza, co tworzy podpis cyfrowy DS. Dzięki temu podpis cyfrowy jest ściśle powiązany z wiadomością i kluczem prywatnym nadawcy. Nadawca wysyła oryginalną wiadomość M oraz uzyskany podpis cyfrowy DS. [9]



RYSUNEK 3.1. Schemat tworzenia podpisu cyfrowego po stronie nadawcy

Odbiorca, musi przejść odwrotne kroki, przedstawione na Rysunku 3.2. Używając publicznego klucza nadawcy, odszyfrowuje podpis DS i porównuje wynikowy skrót MD1 z własnoręcznie obliczonym skrótem MD2 z otrzymanej wiadomości M. Jeśli skróty się zgadzają, podpis jest

uznawany za ważny. Oznacza to, że wiadomość pochodzi od nadawcy i nie została zmieniona po jej podpisaniu.



RYSUNEK 3.2. Schemat potwierdzania autentyczności wiadomości przez podpis cyfrowy

Bezpieczeństwo algorytmu RSA opiera się na trudności faktoryzacji dużych liczb. Aktualne systemy kryptograficzne uznają tę operację za obliczeniowo niewykonalną w praktycznym czasie dla kluczy o odpowiednio dużych rozmiarach, na przykład 2048-bitowych. Algorytm generowania pary kluczy RSA w perspektywie obliczeń komputerowych jest prosty i wygląda następująco:

1. Wybranie 2 liczby pierwsze: p i q
2. Obliczenie $N = p * q$
3. Wyliczenie $\phi(n) = (p - 1)(q - 1)$
4. Wybranie wykładnika publicznego e spełniającego warunki: $1 < e < \phi(n)$ oraz $\gcd(e, \phi(n)) = 1$, co oznaczam, że największy wspólny dzielnik liczb e oraz $\phi(n) = 1$
5. Obliczenie prywatnego wykładnika $d \equiv e^{-1} \bmod \phi(n)$

W wyniku tych obliczeń uzyskujemy klucz publiczny (e, n) oraz klucz prywatny (d, n) .

3.2 Algorytm Shora

Algorytm Shora, opracowany przez Petera Shora w 1994 roku [10], jest algorytmem kwantowym, a dokładniej zbiorem trzech algorytmów opisujących rozwiązanie problemów: faktoryzacji liczb pierwszych, problemu logarytmów dyskretnych oraz problemu znajdowania okresowości. Jednak zazwyczaj terminu *Algorytm Shora* używa się w kontekście kwantowego algorytmu faktoryzacji liczb pierwszych. Algorytm ten wykorzystuje zasady mechaniki kwantowej, w szczególności superpozycję i splątanie kwantowe, do rozwiązywania problemów faktoryzacji znacznie szybciej niż klasyczne algorytmy. Dla liczby N , klasyczne metody faktoryzacji mają złożoność wykładniczą, podczas gdy algorytm Shora może rozwiązać ten problem w czasie $O((\log N)^3)$. Algorytm Shora powoduje duże zagrożenie dla znanych nam obecnie metod kryptografii klucza publicznego, bazujących na problemie faktoryzacji liczb pierwszych takich jak RSA, ale także algorytmów wymiany klucza w skończonym polu Diffiego-Helmana lub wymiany kluczy przy wykorzystaniu krzywych eliptycznych.

Algorytm Shor'a podzielony jest na 4 główne etapy:

1. Wybierz losowo liczbę całkowitą g , taką że $g < N$ oraz nie posiadającą wspólnych czynników z poszukiwaną liczbą N
2. Znajdź wykładnik r taki, że $g^r = mN + 1$, gdzie m - liczba całkowita
3. Oblicz $(g^{\frac{r}{2}} + 1)$ oraz $(g^{\frac{r}{2}} - 1)$ (tylko gdy r jest liczbą parzystą, w innym przypadku należy wrócić do kroku 1)
4. Użyj algorytmu Euklidesa do wyznaczenia największego wspólnego dzielnika i obliczenia składowych p i q liczby N

Każdy z tych etapów można z powodzeniem wykonać na klasycznych komputerach, ale kluczową przewagą komputerów kwantowych w algorytmie Shora jest krok 2. Wykorzystując superpozycję i mechanikę kwantową odnalezienie wykładnika r staje się nieporównywalnie szybsze przy wykorzystaniu komputerów kwantowych. Algorytm Shora wykorzystuje bardzo prostą mechanikę okresowości w teorii liczb, mówiącą, że każda liczba g (taka że $g < N$ oraz $\gcd(g, N) = 1$) podniesiona do potęgi r ostatecznie, a co najważniejsze w kontekście algorytmu Shora, okresowo będzie dawała wynik równy $mN + 1$. Szacowana, asymptotyczna ilość kubitów wymaganych do przeprowadzenia algorytmu Shora różni się w zależności od przeprowadzonych badań i szacunków. W przykładzie poniżej posłużę się szacunkiem z pracy [11] zakładającym posiadanie około 4100 kubitów. Wykorzystanie mechaniki kwantowej w kroku 2 polega na utworzeniu 2 zbiorów kubitów:

- $|x\rangle = |0\rangle + |1\rangle + |2\rangle + \dots + |10^{1234}\rangle$ w rozmiarze 4100 kubitów
- $|w\rangle = |0\rangle|0\rangle|0\rangle\dots|0\rangle$ w rozmiarze 2050 kubitów

Następnie posiadając wybraną wartość g , przeprowadzone zostaje obliczenie:

$$|\frac{g^0}{N}\rangle|\frac{g^1}{N}\rangle|\frac{g^2}{N}\rangle\dots|\frac{10^{1234}}{N}\rangle$$

z którego do drugiej grupy kubitów $|w\rangle$ zapisane zostają reszty z poszczególnych operacji:

$$|w\rangle = |rem_0\rangle|rem_1\rangle|rem_2\rangle\dots$$

Wykonując te operacje uzyskujemy splątany stan systemu kwantowego łączący obie te grupy kubitów (superpozycje) $|x\rangle$ oraz $|w\rangle$:

$$|0\rangle|rem_0\rangle + |1\rangle|rem_1\rangle + |2\rangle|rem_2\rangle + \dots$$

Niestety ze względu na działanie mechaniki kwantowej, odczytanie wartości z takiej superpozycji jest losowe i niemożliwe jest uzyskanie pełnego stanu. Jednak wykorzystując wiedzę, że wartość reszty z operacji $|\frac{g^x}{N}\rangle$ jest okresowa możemy dokonać pomiaru superpozycji dla danej wartości reszty, tym samym uzyskując:

$$|a\rangle|rem\rangle + |b\rangle|rem\rangle + |c\rangle|rem\rangle + \dots$$

Kluczowym jest uzyskanie wartości wykładnika r w działaniu $g^r = mN + 1$, a jest to możliwe ponieważ nieważne dla jakiej reszty uzyskamy wartość okresu, w którym ta reszta się powtarza, będzie ona taka sama dla każdej wartości reszty. Na tej podstawie można zapisać:

$$|a\rangle|rem\rangle + |a+r\rangle|rem\rangle + |a+2r\rangle|rem\rangle + \dots$$

$$|a\rangle + |a+r\rangle + |a+2r\rangle + \dots$$

Posiadając taką superpozycję okresowych stanów, możemy przeprowadzić obliczenia wykorzystując Kwantową Transformację Fouriera, w wyniku uzyskując zbiór stanów:

$$|c_1 \frac{1}{r}\rangle + |c_2 \frac{1}{r}\rangle + |c_3 \frac{1}{r}\rangle + \dots$$

Posiadając wartość r , możemy przejść do następnych kroków algorytmu Shora, w których obliczamy $(g^{\frac{r}{2}} + 1)$ oraz $(g^{\frac{r}{2}} - 1)$, a następnie przy wykorzystaniu algorytmu Euklidesa znajdujemy składowe p i q liczby N .

Wszystkie te obliczenia zakładają występowanie w systemie perfekcyjnych kubitów, które nie posiadają zakłóceń i nie wprowadzają błędów. W realnych zastosowaniach konieczne jest wykorzystanie dodatkowych kubitów korekcyjnych i uwzględnienia wpływu szumu kwantowego. Według danych firmy IBM na rok 2023 [12], obecny stabilny komputer kwantowy posiada 128 kubitów, ale plany na przyszłe lata zakładają uzyskanie komputera kwantowego posiadającego 2000 fizycznych kubitów, około jednego miliarda bramek kwantowych oraz w pełni działających systemów korekcji błędów. Jeśli komputery kwantowe osiągną odpowiednią skalę i stabilność, będą w stanie efektywnie wykorzystywać algorytm Shora do faktoryzacji dużych liczb używanych w kluczach RSA. To spowodowałoby, że obecne systemy oparte na algorytmie RSA stałyby się podatne na ataki, ponieważ klucze prywatne mogłyby zostać złamane w praktycznie realnym czasie. W konsekwencji integralność i autentyczność podpisów cyfrowych mogłyby być zagrożone, co stanowi poważne wyzwanie dla całego systemu bezpieczeństwa cyfrowego. Dlatego tak istotne jest jak najszybsze opracowywanie standardów i algorytmów odpornych na potencjalne zagrożenia ze strony komputerów kwantowych. Obecnie wykorzystanie algorytmu Shora ogranicza się do faktoryzacji maksymalnie liczby 21 z wykorzystaniem czysto kwantowego systemu, bez preprocesowania przez klasyczne komputery, w celu zmniejszenia liczby wymaganych kubitów. W obliczu zagrożeń wynikających z postępu w dziedzinie komputerów kwantowych, społeczność kryptograficzna aktywnie pracuje nad rozwojem algorytmów odpornych na ataki kwantowe. Obecnie moc obliczeniowa komputerów kwantowych a dokładnie liczba dostępnych kubitów, nie pozwala na rozwiązywanie skomplikowanych obliczeń związanych z faktoryzacją liczb pierwszych.

3.3 Konkurs NIST

Głównym celem konkursu było zidentyfikowanie i standaryzacja algorytmów kryptograficznych, które mogą zastąpić obecne systemy oparte na problemach matematycznych, takich jak faktoryzacja dużych liczb pierwszych (RSA) czy problem logarytmu dyskretnego (DSA, ECDSA),

które są podatne na ataki przy użyciu komputerów kwantowych. Konkurs NIST składał się oryginalnie z 4 rund selekcyjnych, podczas których zgłoszone algorytmy były poddawane szczegółowej analizie i ocenie pod kątem bezpieczeństwa, wydajności oraz praktyczności implementacji. W pierwszej rundzie, zakończonej w 2019 roku, NIST otrzymał 69 zgłoszeń, z których wybrał 26 kandydatów do dalszych etapów [13]. W drugiej rundzie, zakończonej w 2020 roku, liczba kandydatów została zredukowana do 15 algorytmów, podzielonych na dwie kategorie: szyfrowanie i wymiana kluczy oraz podpisy cyfrowe. Trzecia runda, która rozpoczęła się w 2021 roku, miała na celu dalszą analizę i testowanie pozostałych kandydatów, aby wyłonić najbardziej obiecujące rozwiązania. Ostatecznie w procesie konkursu wybrane zostały 4 algorytmy: *CRYSTALS-KYBER* jako algorytm uzgadniania klucza oraz *CRYSTALS-Dilithium* jako algorytm podpisu cyfrowego [14]. Oprócz tego w kategorii podpisu cyfrowego do standardu zostały dodane 2 dodatkowe algorytmy: *FALCON*, ze względu na zbyt duży rozmiar podpisów w algorytmie *CRYSTALS-Dilithium* dla niektórych zastosowań oraz *SPHINCS⁺* jako alternatywę do algorytmów opartych na kratkach. Pomimo wyboru finalistów i ogłoszenia publicznych szkiców standardów FIPS 203-205[15], ze względu na wyłonienie tylko jednego kandydata w dziedzinie szyfrowania i wymiany kluczy, NIST ogłosił czwartą rundę konkursu, w której dalej rozważane są algorytmy: *BIKE*, *HQC*, *SIKE* oraz *Classic McEliece* [16].

Jednym z najbardziej wyróżniających się podejść w konkursie NIST była kryptografia oparta na kratkach (ang. *lattice-based cryptography*). Algorytmy te wykorzystują trudne problemy matematyczne związane z kratami geometrycznymi, których zastosowanie w algorytmach kryptograficznych jest uważane za odporne na ataki zarówno klasycznych, jak i kwantowych komputerów. Podczas trwania konkursu, NIST monitorował również rozwój innych podejść kryptograficznych, takich jak kryptografia oparta na kodzie (ang. *code-based cryptography*) czy kryptografia wielowymiarowa. Każde z tych podejść miało swoje unikalne właściwości i potencjalne zalety, jednak to kryptografia oparta na kratkach zdominowała konkurs, ze względu na swoją solidność i wszechstronność.

3.4 Kryptografia oparta na kratkach

Kryptografia oparta na kratkach jest jednym z najbardziej obiecujących obszarów badawczych w dziedzinie kryptografii postkwantowej. Oparta na trudnych problemach matematycznych związanych z kratami geometrycznymi w przestrzeni wielowymiarowej, kryptografia oparta na kratkach oferuje potencjalnie bezpieczne rozwiązania w erze postkwantowej, gdzie tradycyjne metody kryptograficzne mogą być podatne na ataki kwantowe.

Krata (ang. *lattice*) to okresowy zbiór punktów w nieskończonej przestrzeni wielowymiarowej, który można opisać za pomocą wektorów bazowych. Jest to abstrakcyjna struktura matematyczna, która służy jako fundament dla wielu algorytmów kryptograficznych. Kluczowym elementem algebry liniowej krat jest fakt, że taką samą kratę można stworzyć za pomocą różnych wektorów bazowych. Powoduje to występowanie dobrych (ang. *good basis*) i złych (ang.

bad basis) wektorów, za pomocą których można utworzyć kratę. Złe wektory to takie, które w ze względu na dobrane wartości utrudniają osiągnięcie punktów o całkowitych współrzędnych lub który wymagają zaokrąglania w celu uzyskania oczekiwanego wyniku. Wykorzystanie takich wektorów powoduje, że nie wiadomo które wartości zaokrąglać w górę a które w dół. Przy 2-wymiarowej kratce daje to już 4 kombinacje, które znacząco wpływają na prostotę i poprawność otrzymanego wyniku. Zwiększając liczbę wymiarów kraty np. do 100 wymiarów, liczba kombinacji takich przybliżeń rośnie wykładniczo osiągając w przybliżeniu 10^3 , znacząco utrudniając uzyskanie poprawnego lub chociaż bliskiego wyniku.

Wykorzystując wektory bazowe $B = \begin{pmatrix} 6 \\ 14 \end{pmatrix}, \begin{pmatrix} 3 \\ 8 \end{pmatrix}$ i próbując uzyskać wartości a i b , w równaniu poszukując punktu $(11.6, 4.2)$ [17]:

$$a \begin{pmatrix} 6 \\ 14 \end{pmatrix} + b \begin{pmatrix} 3 \\ 8 \end{pmatrix} \approx \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix} \quad (3.1)$$

$$a = 13.4, b = -22.9 \quad (3.2)$$

$$13.4 \begin{pmatrix} 6 \\ 14 \end{pmatrix} - 22.9 \begin{pmatrix} 3 \\ 8 \end{pmatrix} \approx \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix} \quad (3.3)$$

Przy zaokrągleniu wartości a do $a \approx 13$ oraz $b \approx -23$, otrzymujemy punkt $(9, -2)$ który dalece odbiega od oczekiwanego punktu. Zmieniając zaokrąglenia poszczególnych współrzędnych będziemy uzyskiwać podobnie dalekie wyniki.

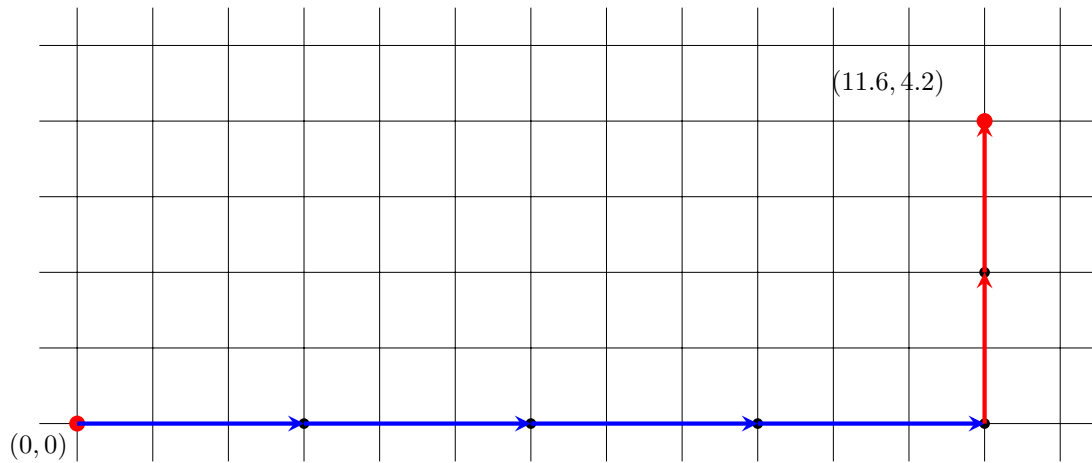
Natomiast wykorzystując podstawę $B = \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix}$:

$$a \begin{pmatrix} 3 \\ 0 \end{pmatrix} + b \begin{pmatrix} 0 \\ 2 \end{pmatrix} \approx \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix} \quad (3.4)$$

$$a = 3.86, b = 2.1 \quad (3.5)$$

$$4 \begin{pmatrix} 3 \\ 0 \end{pmatrix} + 2 \begin{pmatrix} 0 \\ 2 \end{pmatrix} \approx \begin{pmatrix} 12 \\ 4 \end{pmatrix} \quad (3.6)$$

osiągamy o wiele bardziej obiecujący i przypominający poszukiwany punkt wynik. Dlatego tak istotne jest wykorzystanie dobrych wektorów. Na Rysunku 3.3. za pomocą grupy wektorów $B = \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix}$ została stworzona krata z tym samym rozmieszczeniem punktów co krata stworzona używając wektorów $B = \begin{pmatrix} 6 \\ 14 \end{pmatrix}, \begin{pmatrix} 3 \\ 8 \end{pmatrix}$, której rysunek jest za duży żeby go tutaj prezentować.

RYSUNEK 3.3. Przykładowa krata dwuwymiarowa stworzona z wektorów $(3,0)$ oraz $(0,2)$

3.4.1 Problemy i schematy w kryptografii opartej na kratkach

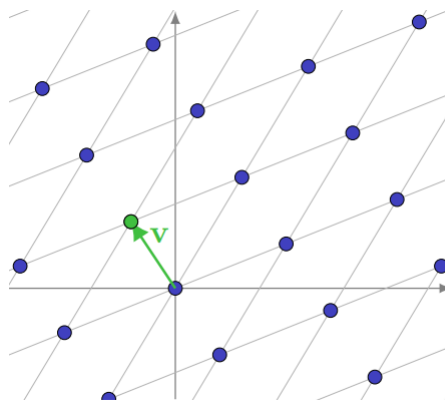
Kluczowymi problemami i schematami matematycznymi wykorzystywanymi w kryptografii opartej na kratkach są problemy: *Shortest Vector Problem* (SVP), *Closest Vector Problem* (CVP), *Shortest Integer Solution* (SIS), *Learning With Errors* (LWE) oraz inne ich pochodne np.: *Shortest Independent Vector Problem* (SIVP), *Ring Learning With Errors* (RLWE).

SVP

Problem najkrótszego wektora w algebrze krat można zdefiniować jako:

Twierdzenie 1 Mając bazę $\mathbf{B}$ kraty $\mathcal{L} = \mathcal{L}(\mathbf{B})$, znajdź niezerowy wektor $\mathbf{v}$, taki że $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$ [18]

co w praktyce oznacza odnalezienie najkrótszego wektora w przestrzeni macierzy, co zostało pokazane na Rysunku 3.4.



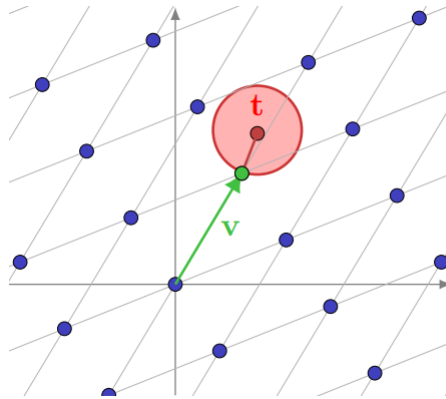
RYSUNEK 3.4. Wizualizacja najkrótszego wektora w kratce [18]

CVP

Problem najbliższego wektora w algebrze krat można zdefiniować jako:

Twierdzenie 2 *Mając bazę B kraty $\mathcal{L} = \mathcal{L}(B)$ oraz wektor docelowy t (niekoniecznie należący do $\mathcal{L}$), znajdź wektor v taki, że $\|v - t\| = \min_{w \in \mathcal{L}} \|w - t\|$ [18]*

co w praktyce oznacza odnalezienie najbliższego wektora do podanego wektora t , który na Rysunku 3.5 został oznaczony jako wektor v .



RYСУNEK 3.5. Wizualizacja najbliższego wektora w kratce [18]

SIS

Problem *Shortest Integer Solution* został opracowany w 1996 i stał się jednym z głównych podstaw działania jednokierunkowych i bezkolizyjnych funkcji skrótu oraz podpisów cyfrowych [19].

Twierdzenie 3 *Mając m uniwersalnie losowych wektorów $a_i \in \mathbb{Z}_q^n$, tworzących macierz $A \in \mathbb{Z}_q^{n \times m}$, znajdź niezerowy wektor całkowity $z \in \mathbb{Z}^m$ o normie $\|z\| \leq \beta$, taki że [20]:*

$$f_A(x) := Ax = 0 \in \mathbb{Z}_q^n$$

Rozwiązanie tego problemu polega na znalezieniu spośród określonej grupy addytywnej, posiadającej dużo losowo wybranych elementów, wystarczająco krótkiej niebanalnej kombinacji całkowitej tych elementów sumującej się do zera.

LWE

Problem *Learning With Errors* został opracowany w 2005 roku i stał się odpowiednikiem problemu SIS dla szyfrowania, przy czym oba problemy są do siebie logicznie podobne [19]. LWE dzieli się na 2 podstawowe zagadnienia:

- *Search* – $LWE_{n,q,\chi,m}$:

Twierdzenie 4 Mając m niezależnych próbek $(a_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ pobranych z $A_{s,\chi}$ dla jednostajnie losowego $s \in \mathbb{Z}_q^n$ znajdź s . [20]

- *Decision* – $LWE_{n,q,\chi,m}$:

Twierdzenie 5 Mając niezależnych próbek $(a_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, gdzie każda próbka jest rozłożona zgodnie z jednym z następujących rozkładów:

1. $A_{s,\chi}$ dla jednostajnie losowego $s \in \mathbb{Z}_q^n$ (ustalonego dla wszystkich próbek), lub
2. rozkładem jednostajnym,

rozróżnij, który przypadek ma miejsce. [20]

Wykorzystanie LWE w kryptografii można pokazać na prostym modelu systemu wymiany informacji. Korzystając z podstawowej formy LWE:

$$A \times s + e = t, \quad A \in \mathbb{Z}^{(d \times n)}, \quad s \in \mathbb{Z}^{(n \times 1)}, \quad e, t \in \mathbb{Z}^{(d \times 1)} \quad (3.7)$$

gdzie:

- A - macierz wektorów bazowych o rozmiarze $d \times n$
- s - n -kolumnowy wektor
- e, t - d -kolumnowe wektory

można skonstruować system gdzie macierz wektorów bazowych A oraz wektor docelowy t są traktowane jako składowe klucza publicznego, a wektor sekretu s jest prywatny. Proces szyfrowania można przedstawić jako:

$$u = A \cdot x = A^T \times x, \quad (3.8)$$

$$A \in \mathbb{Z}^{(n \times d)}, \quad x \in \mathbb{Z}^{(d \times 1)}, \quad u \in \mathbb{Z}^{(n \times 1)}$$

$$v = t \cdot x + m = t^T \times x + m, \quad (3.9)$$

$$t^T \in \mathbb{Z}^{(1 \times d)}, \quad x \in \mathbb{Z}^{(d \times 1)}, \quad m, \quad v \in \mathbb{Z}^{(l \times 1)}$$

Wiadomość m szyfruje się z wykorzystaniem dwóch wektorów kolumnowych: u oraz v oraz losowego wektora x .

Proces odkodowywania szyfrogramu (u, v) wiadomości m jest bardziej skomplikowany i wygląda następująco:

$$z = u \cdot s = u^T \times x, \quad (3.10)$$

$$u^T \in \mathbb{Z}^{(1 \times n)}, \quad s \in \mathbb{Z}^{(n \times 1)}, \quad z \in \mathbb{Z},$$

$$\begin{aligned} v' &= v - z = t \cdot x + m - u \cdot s \\ &= t^T \times x + m - u^T \times s \\ &= (A \times s + e)^T \times x + m - (A \times x)^T \times s \\ &= (A \times s)^T \times x + e^T \times x + m - x^T \times (A^T)^T \times s \\ &= e^T \times x + m \end{aligned} \quad (3.11)$$

$$x^T \in \mathbb{Z}^{(1 \times d)}, \quad A \in \mathbb{Z}^{(d \times n)}, \quad s \in \mathbb{Z}^{(n \times 1)}, \quad e^T \in \mathbb{Z}^{(1 \times d)}, \quad x \in \mathbb{Z}^{(d \times 1)}, \quad m \in \mathbb{Z}^{(1 \times 1)}$$

Ostatecznie otrzymując wiadomość m :

$$\begin{aligned} m &= v' - e^T \times x = e^T \times x - m - e^T \times x = m, \\ e^T &\in \mathbb{Z}^{(1 \times d)}, \quad x \in \mathbb{Z}^{(d \times 1)}, \quad m \in \mathbb{Z}^{(1 \times 1)} \end{aligned} \quad (3.12)$$

Do odszyfrowania wiadomości odbiorca musi posiadać klucz publiczny oraz przybliżony wektor błędu e . Uzyskanie podobnego wektora błędu jest możliwe, ponieważ jest on wybierany losowo lub pseudolosowo na postawie rozkładu Gaussa używając małych wartości (inaczej niemożliwe byłoby uzyskanie poprawnego wyniku).

MLWE

W perspektywie algorytmu *CRYSTALS-Dilithium* kluczową modyfikacją jest uogólnienie techniki *LWE* oraz *Ring-LWE* do *Module Learning With Errors*. Czerpie ona z wariantu *RLWE* korzystając ze struktur pierścieni, tym samym wykorzystując wydajność tej techniki oraz generalizuje standardową technikę *LWE* wykorzystując moduły. Modyfikacje jakie trzeba wprowadzić względem wspomnianych technik, konieczne do uzyskania *MLWE* można zdefiniować za pomocą twierdzenia:

Twierdzenie 6 *Mając moduł algebraiczny M nad strukturą pierścienia $\mathcal{R}$, publiczną macierz wektorów bazowych $A \in M^{(d \times n)}$ z wartościami $A_{i,j} \in \mathcal{R}$, gdzie $1 \leq i \leq d$, $1 \leq j \leq n$ oraz $d \geq n$, a także publiczny wektor docelowy $t \in M^d$, znajdź wektor sekretu $s \in M^n$, taki że zachodzi równość:*

$$A \times s + e = t$$

Twierdzenie mówi o modyfikacji przestrzeni i założeń tak że, wartości pochodzą z pierścienia $\mathcal{R}$, przy zachowaniu założeń dotyczących np. wektora błędu e , który dalej jest definiowany losowo za pomocą małych wartości.

3.4.2 Wykorzystanie w kryptografii

Wykorzystanie krat i tworzących ich wektorów można wykorzystać w kryptografii np. używając ich jako kluczy. Jako klucz publiczny oznaczana jest zły wektor bazowy, a jako klucz prywatny dobry wektor. Samo szyfrowanie zachodzi w następujący sposób: Chcąc zaszyfrować wartości np. $a = 14$ oraz $b = -24$, które w rzeczywistości mogą oznaczać na przykład litery, posługując się wektorami z przykładów wyżej, należy wykonać następujące obliczenia [17]:

$$14 \begin{pmatrix} 6 \\ 14 \end{pmatrix} - 24 \begin{pmatrix} 3 \\ 8 \end{pmatrix} = \begin{pmatrix} 12 \\ 4 \end{pmatrix} \quad (1)$$

$$\vec{e} = \begin{pmatrix} 0.4 \\ 0.2 \end{pmatrix} \quad (2)$$

$$\begin{pmatrix} 12 \\ 4 \end{pmatrix} + \vec{e} = \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix} \quad (3)$$

W kroku pierwszym następuje zaszyfrowanie naszej wiadomości wykorzystując klucz publiczny $B_{bad} = \begin{pmatrix} 6 \\ 14 \end{pmatrix}, \begin{pmatrix} 3 \\ 8 \end{pmatrix}$. W następnym kroku losowo utworzony zostaje wektor z małymi wartościami, nazywany wektorem błędu. W kroku 3 sumowane zostają wektor wiadomości i wektor błędu uzyskując wartość do przesłania. Dla każdej kolejnej wysłanej wiadomości generowany jest nowy, losowy wektor błędu. Wektor ten musi być wystarczająco mały, żeby nie zaburzyć odczytu punktu, a dokładniej nie zaburzyć zaokrąglenia punktu i odczytania niepoprawnej wartości.

Odszyfrowywanie wiadomości także dzieli się na parę kroków:

$$a \begin{pmatrix} 3 \\ 0 \end{pmatrix} + b \begin{pmatrix} 0 \\ 2 \end{pmatrix} \approx \begin{pmatrix} 11.6 \\ 4.2 \end{pmatrix} \quad (1)$$

$$4 \begin{pmatrix} 3 \\ 0 \end{pmatrix} + 2 \begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 4 \end{pmatrix} \quad (2)$$

$$a \begin{pmatrix} 6 \\ 14 \end{pmatrix} + b \begin{pmatrix} 3 \\ 8 \end{pmatrix} = \begin{pmatrix} 12 \\ 4 \end{pmatrix} \quad (3)$$

$$a = 14, b = -24 \quad (4)$$

W kroku pierwszym obliczany jest najbliższy punkt w kratce na podstawie otrzymanego punktu spoza kraty. Następnie w kroku 2 obliczane są wartości a i b (zaokrąglając je do najbliższej liczby całkowitej). Po obliczeniu najbliższego punktu można podstawić otrzymaną wartość do równania z kluczem publicznym i wyliczyć prawdziwe wartości a i b , uzyskując tym samym wynik.

Rozdział 4

CRYSTALS-Dilithium

CRYSTALS(CRYptographic SuiTe for Algebraic LatticeS)-Dilithium to jedno z dwóch narzędzi, należących do zestawu systemów i schematów *CRYSTALS*. *Dilithium* jest klasycznym schematem podpisów cyfrowych post-kwantowych, opracowanym jako uzupełnienie asymetrycznego kryptosystemu *CRYSTALS-Kyber*. Schemat podpisów cyfrowych *Dilithium* został zaprojektowany z myślą o odporności na ataki kwantowe oraz zapewnieniu bezpieczeństwa obliczeniowego w obliczu przyszłych komputerów kwantowych, wykorzystując trudności obliczeniowe problemu MLWE (ang. *Module-Learning With Errors* jako swoją funkcję pułapki). *Dilithium*, wraz z *Kyber*, reprezentuje zaawansowane narzędzia kryptograficzne, które mają na celu zastąpienie obecnych standardów szyfrowania i podpisów cyfrowych, podatnych na ataki ze strony komputerów kwantowych. Podstawą schematu *Dilithium* jest zaproponowana w 2009 roku przez Vadima Lyubashevsky’ego technika *Fiat-Schamir with Aborts*, [21] skupiająca się na zapewnieniu bezpieczeństwa i zmniejszeniu rozmiaru opartego na kratkach cyfrowego podpisu Fiata-Shamira. Kluczowym aspektem różniącym *Dilithium* chociażby od schematu NTRU, jest wykorzystanie rozkładu jednostajnego w procesie losowego próbkowania, zamiast znanego chociażby z NTRU rozkładu Gaussa. Autorzy zdecydowali się na to ze względu na ciężką i nietrywialną implementację takiej techniki, tym samym przyrównując ten aspekt generowania podpisu do zaproponowanego w 2013 roku schematu Bai-Gailbraith [22].

4.1 Uproszczone kroki

Autorzy podają uproszczony schemat tworzenia i weryfikacji podpisu cyfrowego, który został podzielony na 3 kroki [23]:

Generowanie klucza

Algorytm 1 Algorytm generowania klucza

- 1: $A \leftarrow R_q^{k \times \ell}$
 - 2: $(s_1, s_2) \leftarrow S_\eta^\ell \times S_\eta^k$
 - 3: $t \leftarrow A s_1 + s_2$
 - 4: **return** $(k_{pub} = (A, t), k_{pryw} = (A, t, s_1, s_2))$
-

Algorytm 1 w prostych krokach opisuje proces generowania klucza. Algorytm generuje macierz A o rozmiarze $k \times \ell$. W kroku drugim następuje losowe próbkowanie wektorów sekretu s_1 oraz s_2 . Algorytm zwraca:

- klucz publiczny k_{pub} składający się z macierzy A oraz wyliczonej wartości wektora t
- klucz prywatny k_{pryw} zbudowany na podstawie macierzy A , wektora t oraz dwóch wektorów sekretu s_1 i s_2

Poniżej została zaprezentowana matematyczna notacja przedstawionego algorytmu wraz z przykładem na małych liczbach ilustrującym koncepcyjne działanie algorytmu. Istotny jest fakt, że w rzeczywistości wszystkie operacje zachodzą w wymiarach uniemożliwiających fizyczne przedstawienie obliczeń w sposób czytelny i zrozumiały.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ \vdots & \vdots \\ a_{k1} & a_{k2} \end{bmatrix} \quad (4.1)$$

$$a_{ij} \in \text{Rq} = \mathbb{Z}_q[X]/(X^n + 1)$$

$$s_1 = \begin{bmatrix} s_{11} \\ \vdots \\ s_{l1} \end{bmatrix}, \quad s_2 = \begin{bmatrix} s_{21} \\ \vdots \\ s_{l2} \end{bmatrix} \quad (4.2)$$

$$s_{ij} \in \text{Rq}$$

$$t_1 = A \cdot s_1 = \begin{bmatrix} a_{11}s_{11} + a_{12}s_{12} \\ \vdots \\ a_{k1}s_{11} + a_{k2}s_{12} \end{bmatrix} \quad (4.3)$$

$$t = t_1 + s_2 = \begin{bmatrix} a_{11}s_{11} + a_{12}s_{12} + s_{21} \\ \vdots \\ a_{k1}s_{11} + a_{k2}s_{12} + s_{22} \end{bmatrix} \quad (4.4)$$

$$k_{pub} = \begin{bmatrix} A \\ t \end{bmatrix} \quad (4.5)$$

$$k_{pryw} = \begin{bmatrix} A \\ t \\ s_1 \\ s_2 \end{bmatrix} \quad (4.6)$$

Przykład:

$$A = \begin{bmatrix} 1 + 2X & 3X + 4 \end{bmatrix} \quad (3.1)$$

$$s_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad s_2 = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \quad (3.2)$$

$$t_1 = A \cdot s_1 = \begin{bmatrix} 1 + 2X + 6X + 8 \end{bmatrix} \quad (3.3)$$

$$t = t_1 + s_2 = \begin{bmatrix} 11 + 8X \\ 4 \end{bmatrix} \quad (3.4)$$

$$k_{pub} = \begin{bmatrix} A \\ t \end{bmatrix} = \begin{bmatrix} 1 + 2X & 3X + 4 \\ 11 + 8X & 4 \end{bmatrix} \quad (3.5)$$

$$k_{pryw} = \begin{bmatrix} A \\ t \\ s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} 1 + 2X & 3X + 4 \\ 11 + 8X & 4 \\ 1 & 2 \\ 3 & 4 \end{bmatrix} \quad (3.6)$$

Powyższy przykład pokazuje krok po kroku, jakie operacje są przeprowadzane w poszczególnych etapach algorytmu generowania klucza w algorytmie CRYSTALS-Dilithium, który korzysta z kryptografii opartej na kratkach. Macierz A składa się z elementów należących do pierścienia wielomianów z współczynnikami całkowitymi modulo pewnej liczby pierwszej q . Elementy tego pierścienia mają postać $\mathbb{Z}_q[X]/(X^n+1)$, gdzie q jest liczbą pierwszą, a X^n+1 jest nierozkładalnym wielomianem.

W kroku (3.4), wektor docelowy t jest obliczany jako suma wektorów t_1 , uzyskanego przez mnożenie macierzy A przez jeden z wektorów sekretu oraz drugiego wektora sekretu. Sam mechanizm nie jest skomplikowany i jest wykorzystywany w wielu powszechnie znanych algorytmach, z tą różnicą, że wykorzystuje się tutaj macierze wielomianów. Klucz publiczny k_{pub} składa się z macierzy oraz z wektora docelowego t , natomiast klucz publiczny rozszerzony jest o 2 wektory sekretu s_1 oraz s_2 .

Podpis

Algorytm 2 Algorytm podpisu

```

1:  $z := \perp$ 
2: while  $z = \perp$  do
3:    $y \leftarrow S_{\gamma_1-1}^\ell$ 
4:    $w_1 := HighBits(Ay, 2\gamma_2)$ 
5:    $c \leftarrow H(M \parallel w_1)$ 
6:    $z := y + cs_1$ 
7:   if  $\|z\|_\infty \geq \gamma_1 - \beta$  or  $\|LowBits(Ay - cs_2, 2\gamma_2)\|_\infty \geq \gamma_2 - \beta$  then
8:      $z := \perp$ 
9:   end if
10: end while
11: return  $\sigma = (z, c)$ 

```

Najbardziej zaawansowanym jest algorytm służący do podpisu. Ze względu na dużą liczbę dodatkowych funkcji szczegółowa analiza zostanie przeprowadzana w następnych etapach pracy razem z wyjaśnieniem działania poszczególnych funkcji.

Pierwszym krokiem jest haszowanie wiadomości, polegające na przekształceniu wiadomości M w ciąg znaków o stałej długości, zwany skrótem. W algorytmie *CRYSTALS-Dilithium* używana

jest funkcja skrótu H , która zapewnia, że wszystkie wiadomości o tej samej treści będą miały ten sam skrót. Następnie algorytm pobiera próbkę wektora y z rozkładu prawdopodobieństwa $S_{\gamma_1-1}^\ell$. Rozkład ten jest wybrany w sposób zapewniający bezpieczeństwo algorytmu. Algorytm oblicza następnie wysokie bity iloczynu macierzy A i wektora y . Wysokie bity $HighBits(Ay, 2\gamma_2)$ to najbardziej znaczące bity liczby, a liczba bitów, które są zachowane, jest określana przez parametr $2\gamma_2$.

Wartość c jest obliczana jako wynik funkcji H zastosowanej do połączenia skrótu wiadomości M i najbardziej znaczących bitów $w_1 = HighBits(Ay, 2\gamma_2)$. Funkcja H jest funkcją skrótu lub funkcją przetwarzania, która zapewnia dodatkowe bezpieczeństwo algorytmu. Wektor z jest obliczany jako suma wektora y i iloczynu c oraz wektora klucza tajnego s_1 . Następnie algorytm sprawdza, czy normy $\|z\|_\infty$ i $\|LowBits(Ay - cs_2, 2\gamma_2)\|_\infty$ spełniają określone warunki bezpieczeństwa $\gamma_1 - \beta$ i $\gamma_2 - \beta$.

Weryfikacja podpisu

Algorytm 3 Algorytm weryfikacji podpisu

```

1:  $w'_1 := HighBits(Az - ct, 2\gamma_2)$ 
2:  $valid := \llbracket \|z\|_\infty < \gamma_1 - \beta \rrbracket$  oraz  $\llbracket c = H(M \parallel w'_1) \rrbracket$ 
3: return  $valid$ 

```

Algorytm weryfikuje podpis, sprawdzając, czy wektor z spełnia określone własności matematyczne związane ze skrótem wiadomości M , kluczem publicznym A , i innymi informacjami. Jeśli weryfikacja się nie powiedzie, algorytm powtarza kroki od pobrania próbki wektora y do obliczenia wektora z , aż znajdzie poprawny podpis. Jeśli weryfikacja się powiedzie, algorytm zwraca wartość logiczną $valid$, która informuje o poprawności podpisu.

4.2 Rozszerzona interpretacja

Poniżej przedstawiono parametry wspólne dla wszystkich przedstawionych algorytmów

- n : Wymiar pierścienia wielomianowego R .
- k : Liczba wielomianów w modułach używanych.
- ℓ : Liczba tajnych wielomianów.
- q : Modulo dla współczynników wielomianowych.
- η : Parametr związany z rozkładami szumu, określający zakres klucza tajnego.

Rozszerzony algorytm generowania klucza

Procedura generowania pary kluczy w algorytmie *CRYSTALS-Dilithium* przyjmuje jako wejście parametry całkowitoliczbowe $n \in \mathbb{Z}$, $k \in \mathbb{Z}$, $\ell \in \mathbb{Z}$, $q \in \mathbb{Z}$, $\eta \in \mathbb{Z}$ oraz $d_t \in \mathbb{Z}$:

- d_t : Parametr definiujący kompresję wektora docelowego podczas generowania kluczy.

Parametry k , ℓ i η są ściśle związane z bezpieczeństwem generowanych kluczy asymetrycznych, gdzie pierwsze dwa mogą dostosować złożoność problemu MLWE, podczas gdy ostatni jest kluczowy dla zabezpieczeń poprzez dodanie losowości w procedurze.

Algorytm 4 Rozszerzony algorytm generowania kluczy

Wejście: n, k, ℓ, q, η, d_t
Wyjście: $(k_{\text{pub}}, k_{\text{pryw}})$
Założenia: $n, k, \ell, q, \eta, d_t \in \mathbb{Z}$

- 1: $\zeta \leftarrow \{0, 1\}^n$
- 2: $(\rho, \rho', K) \sim (\{0, 1\}^n \times \{0, 1\}^{2n} \times \{0, 1\}^n) := \mathcal{H}(\zeta)$
- 3: $A \sim R_q^{(k \times \ell)} := \text{Sample}(\rho)$
- 4: $(s, e) \sim (\Upsilon_\eta^\ell \times \Upsilon_\eta^k) := \text{Sample}(\rho')$
- 5: $t := A \cdot s + e$
- 6: $(t_1, t_0) \sim \text{Compress}(t, d_t) := \text{Power2Round}(t, d_t)$
- 7: $t_r := \{0, 1\}^n := H(\rho \parallel t_1)$
- 8: $k_{\text{pub}} := (\rho, t_1)$
- 9: $k_{\text{pryw}} := (\rho, K, t_r, t_0, s, e)$
- 10: **return** $(k_{\text{pub}}, k_{\text{pryw}})$

Wyjściem procedury jest para kluczy asymetrycznych składająca się z klucza publicznego k_{pub} i klucza prywatnego k_{pryw} . Klucz publiczny k_{pub} to para składająca się z losowego (lub pseudo-losowego) ziarna ρ oraz publicznego wektora docelowego t_1 , podczas gdy klucz prywatny k_{pryw} zawiera ziarno ρ , losowe tajne ziarno klucza K , losowy wektor docelowy t_r , tajny wektor s , wektor błędów (inaczej nazywany wektorem szumów) e oraz niski wektor docelowy t_0 . Dodatkowo, publiczny wektor docelowy t jest kompresowany przez określony parametr d_t , definiujący liczbę odrzucanych bitów z tego wektora podczas procedury generowania kluczy.

Rozszerzony algorytm podpisu

Procedura tworzenia cyfrowego podpisu dla określonej wiadomości przyjmuje jako wejście parametry:

- γ_1 : Zakres współczynników dla wektora maskowania y , związanego z rozkładami szumu wykorzystywanymi do maskowania niezbędnego do generowania losowości i unikalności dla każdego produkowanego podpisu.
- γ_2 : Dolny limit zaokrąglenia dla współczynników wielomianowych wektora zobowiązania w , aby zapewnić, że te współczynniki nie są zbyt duże.

- τ : Liczba niezerowych wartości (tj. ± 1 wartości) na wektorze c .
- m : Wiadomość, która ma zostać podpisana.

Algorytm 5 Rozszerzony algorytm podpisu

Wejście: $n, k, \ell, q, \eta, \gamma_1, \gamma_2, \omega, \sigma, \tau, k_{pryw}, m$

Wyjście: σ

Założenia: $n, k, \ell, q, \eta, \gamma_1, \gamma_2, \tau, \omega \in \mathbb{Z}, m \in \{0, 1\}^*$

```

1:  $(\rho, K, t_r, s, e, t_0) \leftarrow k_{pryw}$ 
2:  $A \sim \mathcal{R}_q^{(k \times \ell)} := \text{Sample}(\rho)$ 
3:  $\mu \sim \{0, 1\}^{(2 \times n)} := H(t_r \| m)$ 
4:  $\kappa := 0$ 
5:  $(z, h) := \perp$ 
6:  $\rho' \sim \{0, 1\}^{(2 \times n)} := H(K \parallel \mu)$  ▷ Dla podpisywania losowego
7: while  $(z, h) = \perp$  do
8:    $y \sim \Upsilon_{\gamma_1}^\ell := \text{SampleMask}(\rho, \kappa)$ 
9:    $w := A \times y$ 
10:   $w_1 := \text{HighBits}_q(w, 2 \times \gamma_2)$ 
11:   $\tilde{c} \sim \{0, 1\}^n := H(\mu \parallel w_1)$ 
12:   $c \sim B_\tau := \text{SampleInBall}_\tau(\tilde{c})$ 
13:   $z := y + c \times s$ 
14:   $r_0 := \text{LowBits}_q(w - c \times t_0, 2 \times \gamma_2)$ 
15:   $\beta := \tau \times \eta$ 
16:  if  $\llbracket \|z\|_\infty \geq \gamma_1 - \beta \rrbracket$  or  $\llbracket \|r_0\|_\infty \geq \gamma_2 - \beta \rrbracket$  then
17:     $(z, h) := \perp$ 
18:  else
19:     $h := \text{MakeHint}_q(-c \times t_0, w - c \times e + c \times t_0, 2 \times \gamma_2)$ 
20:    if  $(\llbracket \|c \times t_0\|_\infty \geq \gamma_2 \rrbracket$  or  $\llbracket \text{weight}(h) > \omega \rrbracket)$  then
21:       $(z, h) := \perp$ 
22:    end if
23:  end if
24:   $\kappa := \kappa + \ell$ 
25: end while
26:  $\sigma = (\tilde{c}, z, h)$ 
27: return  $\sigma$ 

```

Wyjściem procedury jest podpis σ składający się z (skompresowanego) wektora $\tilde{c}$, wektora docelowego odpowiedzi z oraz wektora wskazówki h . Wektor $\tilde{c}$ ma na celu poprawę efektywności podpisu σ pod względem jego wielkości i transmisji. Z wektora $\tilde{c}$, strona weryfikująca podpis σ będzie mogła próbkować i rozpakowywać go do wektora c . Wektor docelowy odpowiedzi z jest odpowiedzią związaną z wektorem c zgodnie ze strukturą matematyczną problemu MLWE. Następnie wektor wskazówki h ma na celu pomóc drugiej stronie w odzyskaniu pewnych współczynników wielomianowych zobowiązania pochodzącego z podpisu, co zapewnia walidację autentyczności i integralności podpisanej wiadomości. Ta wskazówka zapewnia, że podpis σ jest ważny bez potrzeby przechowywania lub przesyłania dużych ilości nadmiarowych informacji.

Rozszerzony algorytm weryfikacji podpisu

Procedura weryfikacji cyfrowego podpisu dla określonej wiadomości przyjmuje jako wejście parametry całkowitoliczbowe:

- d_t : Wielkość, o jaką zmniejszamy wektor docelowy t_1 podczas kompresji.
- γ_1 : Zakres współczynników dla wektora odpowiedzi z , ustawiający górne ograniczenie na współczynniki wielomianowe.
- γ_2 : Dolne ograniczenie zaokrąglenia współczynników wektora zobowiązania w , aby zapewnić, że te współczynniki nie są zbyt duże.
- τ : Określa liczbę niezerowych wartości (tj. ± 1 wartości) na wektorze c .
- ω : Górne ograniczenie dla wagi Hamminga wektora odpowiedzi h .

Algorytm weryfikacji cyfrowego podpisu dla algorytmu *CRYSTALS-Dilithium* wygląda następująco:

Algorytm 6 Rozszerzony algorytm weryfikacji podpisu

Wejście: $n, k, \ell, q, \eta, \gamma_1, \gamma_2, \omega, d_t, \sigma, \tau, k_{pub}, m, \sigma$

Wyjście: *valid*

Założenia: $n, k, \ell, q, \eta, \gamma_1, \gamma_2, \tau, \omega \in \mathbb{Z}, m \in \{0, 1\}^*$

- 1: $(\rho, t_1) \leftarrow k_{pub}$
 - 2: $A \sim R_q^{(k \times \ell)} := \text{Sample}(\rho)$
 - 3: $\mu \sim \{0, 1\}^{(2 \times n)} := H(H(\rho \parallel t_1) \parallel m)$
 - 4: $c := \text{SampleInBall}_\tau(\tilde{c})$
 - 5: $w'_1 := \text{UseHint}_q(h, A \times z - c \times t_1 \times 2^{d_t}, 2 \times \gamma_2)$
 - 6: $\beta := \tau \times \eta$
 - 7: $\text{valid} := (\|z\|_\infty < \gamma_1 - \beta) \text{ or } [\tilde{c} = H(\mu \| w'_1)] \text{ and } [\text{weight}(h) \leq \omega]$
 - 8: **return** valid
-

Ten algorytm sprawdza poprawność cyfrowego podpisu σ dla danej wiadomości m i klucza publicznego k_{pub} . Sprawdza, czy wszystkie trzy warunki logiczne są spełnione jednocześnie: 1. Współczynniki wielomianowe wektora odpowiedzi z nie przekraczają górnego ograniczenia $\gamma_1 - \beta$, gdzie $\beta = \tau \times \eta$. 2. Wynik skrótu funkcji kryptograficznej $H(\mu \| w'_1)$ jest równy (skompresowanemu) wektorowi $\tilde{c}$ wyodrębnionemu z podpisu σ . 3. Waga Hamminga wektora odpowiedzi h wyodrębnionego z podpisu σ nie przekracza górnego ograniczenia ω .

Jeśli wszystkie te trzy warunki są spełnione, weryfikacja cyfrowego podpisu σ dla danej wiadomości m jest udana, a podpis jest uznawany za prawidłowy.

4.3 Algorytmy pomocnicze

W algorytmie *CRYSTALS-Dilithium* wykorzystuje się szereg algorytmów pomocniczych, które odgrywają kluczową rolę w realizacji głównych procedur: generowania kluczy, tworzenia podpi-

sów oraz ich weryfikacji. Algorytmy te odpowiadają za różnorodne zadania, takie jak generowanie losowych wektorów, przekształcanie danych wejściowych oraz wykonywanie obliczeń niezbędnych do zachowania bezpieczeństwa i wydajności całego schematu kryptograficznego. Poniżej znajdują się szczegółowe opisy i implementacje najważniejszych algorytmów pomocniczych stosowanych w *CRYSTALS-Dilithium*.

SampleInBall

Algorytm 7 SampleInBall

```

1: Initialize:  $c = c_0c_1\dots c_{255} = 00\dots 0$ 
2:   for  $i = 256 - \tau; i \leq 255$ 
3:      $j \leftarrow \{0, 1, \dots, i\}$ 
4:      $s \leftarrow \{0, 1\}$ 
5:      $c_i := c_j$ 
6:      $c_j := (-1)^s$ 
7: return  $c$ 

```

Algorytm *SampleInBall* generuje losowy wektor c należący do zbioru B_τ , który składa się z elementów pierścienia wielomianowego R z τ współczynnikami wynoszącymi -1 lub 1 , a pozostałymi $256 - \tau$ współczynnikami równymi 0 . Początkowo wektor c jest wektorem zerowym, a następnie jego elementy są modyfikowane, aby osiągnąć wymaganą strukturę w następujący sposób:

1. Wylosuj indeks j z zakresu $\{0, 1, \dots, i\}$.
2. Wylosuj wartość s spośród $\{0, 1\}$.
3. Ustaw wartość c_j na c_i .
4. Ustaw wartość $(-1)^s$ na c_j .

Po zakończeniu pętli, wektor c ma dokładnie τ elementów wynoszących -1 lub 1 , a reszta elementów wynosi 0 . Algorytm ten jest stosowany w schemacie podpisu cyfrowego, aby zapewnić losowość i unikalność podpisów. Służy do generowania wektora c , który jest trudny do odgadnięcia, co wzmacnia bezpieczeństwo kryptograficzne. Ten proces jest kluczowy dla zachowania integralności i niezawodności schematu podpisu, zwłaszcza w kontekście problemu MLWE (Learning With Errors opartego na siatkach).

Decompose

Algorytm 8 Decompose

```

1:  $r := r \bmod^+ q$ 
2:  $r_0 := r \bmod^\pm \alpha$ 
3: if  $(r - r_0 == q - 1)$  then
4:    $r_1 := 0$ 
5:    $r_0 := r_0 - 1$ 
6: else
7:    $r_1 := (r - r_0)/\alpha$ 
8: end if
9: return  $(r_1, r_0)$ 

```

Algorytm *Decompose* służy do podziału liczby r na dwie części: r_1 i r_0 . Jego celem jest kontrolowanie rozkładu liczby r w kontekście kryptografii, aby zapobiec niepożądanym zmianom w jej wyższych bitach. Na początku, liczba r jest przekształcana do dodatniej reszty z dzielenia przez q . Następnie, używając dodatkowego parametru α , obliczana jest reszta modulo α . Wynikowy r_0 mieści się zawsze w przedziale od $-\frac{\alpha}{2}$ do $\frac{\alpha}{2}$.

Algorytm sprawdza warunek, czy różnica między r a r_0 wynosi $q - 1$. Jeśli tak, r_1 jest ustawiane na 0, co oznacza, że r_1 nie wnosi wkładu do wyniku, a r_0 jest zmniejszane o 1. Jeśli warunek graniczny nie jest spełniony, r_1 jest obliczane jako iloraz różnicy r minus r_0 przez α . Na końcu, algorytm zwraca parę (r_1, r_0) , reprezentującą podział liczby r .

HighBits i LowBits

Algorytmy *HighBits* i *LowBits* są kluczowymi podprocedurami w głównym algorytmie podpisu, które umożliwiają operowanie na wyższych i niższych bitach elementów w $\mathbb{Z}_q$.

Algorytm 9 HighBits

```

1:  $(r_1, r_0) := \text{Decompose}_q(r, \alpha)$ 
2: return  $r_1$ 

```

Algorytm 10 LowBits

```

1:  $(r_1, r_0) := \text{Decompose}_q(r, \alpha)$ 
2: return  $r_0$ 

```

W algorytmie podpisu algorytmy *HighBits* i *LowBits* biorą udział w zmniejszeniu rozmiaru klucza publicznego, konieczne jest wyodrębnienie "wyższych" i "niższych" bitów elementów w $\mathbb{Z}_q$.

MakeHint i UseHint

Algorytm 11 MakeHint

```

1:  $r_1 := \text{HighBits}_q(r, \alpha)$ 
2:  $v_1 := \text{HighBits}_q(r + z, \alpha)$ 
3: return  $\llbracket r_1 \neq v_1 \rrbracket$ 

```

Algorytm 12 UseHint

```

1:  $m := (q - 1)/\alpha$ 
2:  $(r_1, r_0) := \text{Decompose}_q(r, \alpha)$ 
3: if  $(h == 1 \text{ and } r_0 > 0)$  then
4:   return  $(r_1 + 1) \bmod^+ m$ 
5: end if
6: if  $(h == 1 \text{ and } r_0 \leq 0)$  then
7:   return  $(r_1 - 1) \bmod^+ m$ 
8: end if
9: return  $r_1$ 

```

Algorytm *MakeHint* porównuje wyższe bity liczby r i $r + z$, gdzie z jest małym elementem, służącym do generowania elementu *hint* informującego o ewentualnej zmianie. Wartość zwracana przez ten algorytm jest równa 1, jeśli bity r_1 i v_1 są różne, w przeciwnym razie jest równa 0.

Algorytm *UseHint* wykorzystuje podział liczby r na wyższe r_1 i niższe r_0 bity za pomocą algorytmu *Decompose*. Na podstawie otrzymanej wskazówki h , określa, czy dodać lub odjąć 1 od r_1 modulo m , aby odtworzyć wyższe bity sumy $r + z$.

Power2Round

Algorytm 13 Power2Round

```

1:  $r := r \bmod^+ q$ 
2:  $r_0 := r \bmod^\pm 2^d$ 
3: return  $((r - r_0)/2^d, r_0)$ 

```

Algorytm *Power2Round* jest używany do podziału liczby r z ciała $\mathbb{Z}_q$ na jej "wyższe" i "niższe" bity w sposób bitowy. Pierwszym krokiem jest redukcja $r \bmod q$, co zapewnia, że liczba r znajduje się w przedziale od 0 do $q - 1$. Następnie obliczana jest reszta r_0 z dzielenia r przez 2^d , gdzie d jest ustalonym parametrem. Wynikiem działania algorytmu jest para liczb $((r - r_0)/2^d, r_0)$, reprezentująca rozkład liczby r na dwie części:

- $(r - r_0)/2^d$ - część zawierająca wyższe bity liczby r ,
- r_0 - część zawierająca niższe bity liczby r .

Algorytm *Power2Round* jest używany w kontekście kryptografii do precyzyjnego zarządzania bitami liczbowymi, co minimalizuje ryzyko niepożądanych zmian w wyższych bitach liczby r , szczególnie w sytuacjach, gdy dodawane są małe liczby do r .

Pozostałe algorytmy

- *SampleMask*: odgrywa kluczową rolę w procesie podpisywania poprzez generowanie wektora maskującego y , którego wielomiany posiadają współczynniki mieszczące się w przedziale określonym przez γ_1 . Parametr ten jest wybierany strategicznie: musi być wystarczająco duży, aby zachować poufność klucza prywatnego (zapewniając, że algorytm podpisywania pozostaje zero-wiedzy), ale jednocześnie wystarczająco mały, aby uniemożliwić łatwe fałszowanie podpisów
- *Compress*: zmniejsza rozmiar publicznego wektora docelowego t za pomocą parametru d_t , który określa długość bitową wykorzystywaną podczas kodowania i generowania klucza.
- *Decompress*: jest to odwrotność algorytmu *Compress*, który przywraca oryginalny wektor t z jego skompresowanej postaci.

4.4 Przykładowy przebieg algorytmu CRYSTALS-Dilithium

Parametry wejściowe

- $n = 8$ (długość ziarna ζ)
- $k = 2$ (liczba wierszy macierzy A)
- $\ell = 2$ (liczba kolumn macierzy A)
- $q = 17$ (modulo dla elementów macierzy A i wektorów s, e, t)
- $\eta = 3$ (parametr do losowania wektorów s i e)
- $d_t = 2$ (parametr kompresji t)

4.4.1 Algorytm generowania klucza

Krok 1: Generowanie ziarna losowości ζ

$$\zeta = 11001001$$

Krok 2: Funkcja hashująca $\mathcal{H}$

$$(\rho, \rho', K) \sim \mathcal{H}(\zeta)$$

$$\rho = 10011010, \quad \rho' = 0110001101010100, \quad K = 11100100$$

Krok 3: Losowanie macierzy A

$$A \sim R_{17}^{(2 \times 2)} := \text{Sample}(\rho)$$

$$A = \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix}$$

Krok 4: Losowanie wektorów s i e

$$(s, e) \sim (\Upsilon_3^2 \times \Upsilon_3^2) := \text{Sample}(\rho')$$

$$s = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \quad e = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Krok 5: Obliczenie wektora t

$$t = A \cdot s + e$$

$$t = \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 26 \\ 37 \end{bmatrix}$$

Krok 6: Kompresja wektora t za pomocą Power2Round

$$(t_1, t_0) \sim \text{Power2Round}(t, 2)$$

$$(t_1, t_0) = \text{Power2Round}\left(\begin{bmatrix} 26 \\ 37 \end{bmatrix}, 2\right) = (6, 1)$$

Krok 7: Obliczenie hashu t_r

$$t_r := \{0, 1\}^8 := H(\rho \parallel t_1)$$

$$t_r = H(10011010 \parallel 110) = 01101100$$

Krok 8: Utworzenie klucza publicznego k_{pub}

$$k_{\text{pub}} := (\rho, t_1)$$

$$k_{\text{pub}} = (10011010, 6)$$

Krok 9: Utworzenie klucza prywatnego k_{pryw}

$$k_{\text{pryw}} := (\rho, K, t_r, t_0, s, e)$$

$$k_{\text{pryw}} = (10011010, 11100100, 01101100, 1, \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \end{bmatrix})$$

4.4.2 Algorytm podpisu

Krok 1: Odczytanie klucza prywatnego k_{pryw}

$$(\rho, K, t_r, t_0, s, e) = k_{\text{pryw}}$$

$$\rho = 10011010, \quad K = 11100100, \quad t_r = 01101100, \quad t_0 = 1,$$

$$s = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \quad e = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Krok 2: Generowanie ziarna losowości ζ

$$\zeta = 11001001$$

Krok 3: Funkcja hashująca $\mathcal{H}$

$$(\rho, \rho', K) \sim \mathcal{H}(\zeta)$$

$$\rho = 10011010, \quad \rho' = 0110001101010100, \quad K = 11100100$$

Krok 4: Losowanie macierzy A

$$A \sim R_{17}^{(2 \times 2)} := \text{Sample}(\rho)$$

$$A = \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix}$$

Krok 5: Obliczenie wektora y

$$y := A \cdot s + e$$

$$y = \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 26 \\ 37 \end{bmatrix}$$

Krok 6: Wygenerowanie maski y

$$\text{Maska } y := \text{SampleMask}(y, \gamma_1)$$

$$y = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Krok 7: Obliczenie wektora wysokich bitów w_1

$$w_1 := \text{HighBits}(y, \gamma_2)$$

$$w_1 = \text{HighBits} \left(\begin{bmatrix} 6 \\ 8 \end{bmatrix}, \gamma_2 \right) = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Krok 8: Utworzenie wektora c

$$c := H(\text{wiadomość} \parallel w_1)$$

$$c = H(\text{wiadomosc} \parallel \begin{bmatrix} 3 \\ 4 \end{bmatrix}) = 10101010$$

Krok 9: Wygenerowanie podpisu z

$$z := y + c \cdot s_1$$

$$z = \begin{bmatrix} 6 \\ 8 \end{bmatrix} + 10101010 \cdot \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Krok 10: Utworzenie podpisu σ

$$\sigma := (z, c)$$

4.4.3 Algorytm weryfikacji podpisu**Krok 1: Odczytanie klucza publicznego k_{pub}**

$$(\rho, t_1) = k_{\text{pub}}$$

$$\rho = 10011010,$$

$$t_1 = 6$$

Krok 2: Losowanie macierzy A

$$A \sim R_{17}^{(2 \times 2)} := \text{Sample}(\rho)$$

$$A = \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix}$$

Krok 3: Obliczenie skrótów

$$\mu = H(H(\rho \parallel t_1) \parallel m)$$

$$\mu = H(H(10011010 \parallel 6) \parallel \text{wiadomość})$$

Krok 4: Generowanie wektora $\tilde{c}$

$$\tilde{c} := \text{SampleInBall}_\tau(\mu)$$

$$\tilde{c} = 0011001100110011$$

Krok 5: Użycie wskazówki h w algorytmie UseHint

$$w'_1 := \text{UseHint}_q(h, A \times z - c \times t_1 \times 2^{d_t}, 2 \times \gamma_2)$$

$$w'_1 = \text{UseHint}_q(h, \begin{bmatrix} 4 & 8 \\ 5 & 9 \end{bmatrix} \times \begin{bmatrix} 6 \\ 8 \end{bmatrix} - 0011001100110011 \times 6 \times 2^2, 2 \times 4)$$

Krok 6: Obliczenie β

$$\beta := \tau \times \eta$$

$$\beta := 5 \times 3 = 15$$

Krok 7: Sprawdzenie poprawności podpisu

$$\text{valid} := (\llbracket \|z\|_\infty < \gamma_1 - \beta \rrbracket \text{ or } \llbracket \text{hspace0.5em}\tilde{c} = H(\mu||w'_1) \rrbracket \text{ and } \llbracket \text{weight}(h) \leq \omega \rrbracket)$$

$$\text{valid} := \left(\llbracket \left\| \begin{bmatrix} 6 \\ 8 \end{bmatrix} \right\|_\infty < \gamma_1 - 15 \rrbracket \text{ or } \llbracket 0011001100110011 = H(\mu||w'_1) \rrbracket \text{ and } \llbracket \text{weight}(h) \leq \omega \rrbracket \right)$$

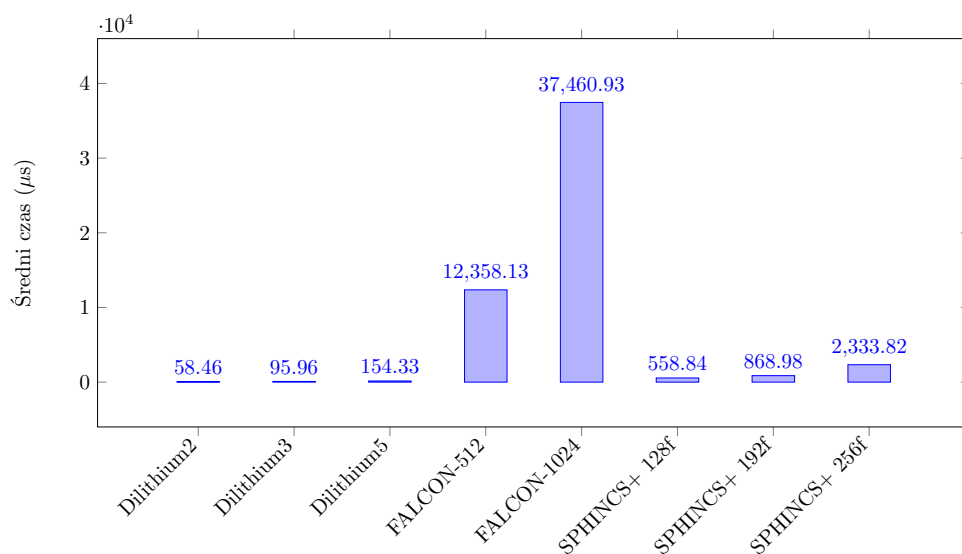
Krok 8: Wynik weryfikacji

$$\text{valid} = \text{true/false}$$

$$\text{valid} = \text{true}$$

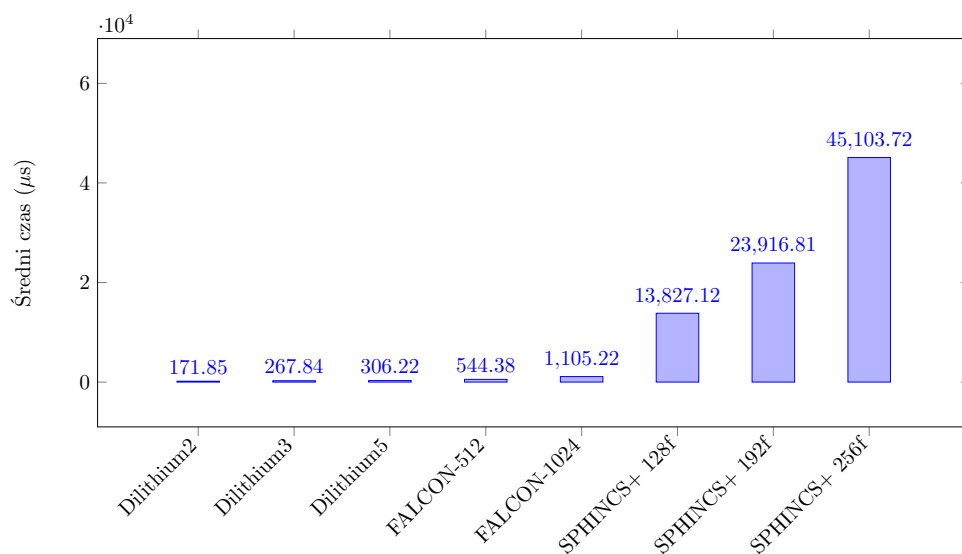
4.5 Porównanie wydajności algorytmów *Dilithium*, *FALCON*, *SPHINCS+*

Rysunek 4.1 przedstawia średni czas wykonania kroku generowania kluczy dla algorytmów Dilithium, FALCON oraz SPHINCS+. Algorytm Dilithium wyróżnia się jako najszybszy w generowaniu kluczy spośród wszystkich trzech kandydatów, co jest istotne w początkowych fazach działania algorytmu. Czasy trwania tego kroku dla algorytmu FALCON są znacząco wyższe, szczególnie dla wariantu z kluczem 1024 bitowym. Algorytm SPHINCS+ również uzyskuje konkurencyjne czasy generowania kluczy, chociaż nieco dłuższe niż w przypadku Dilithium.



RYSUNEK 4.1. Średni czas wykonania kroku generowania kluczy dla algorytmów Dilithium, FALCON oraz SPHINCS+ [24]

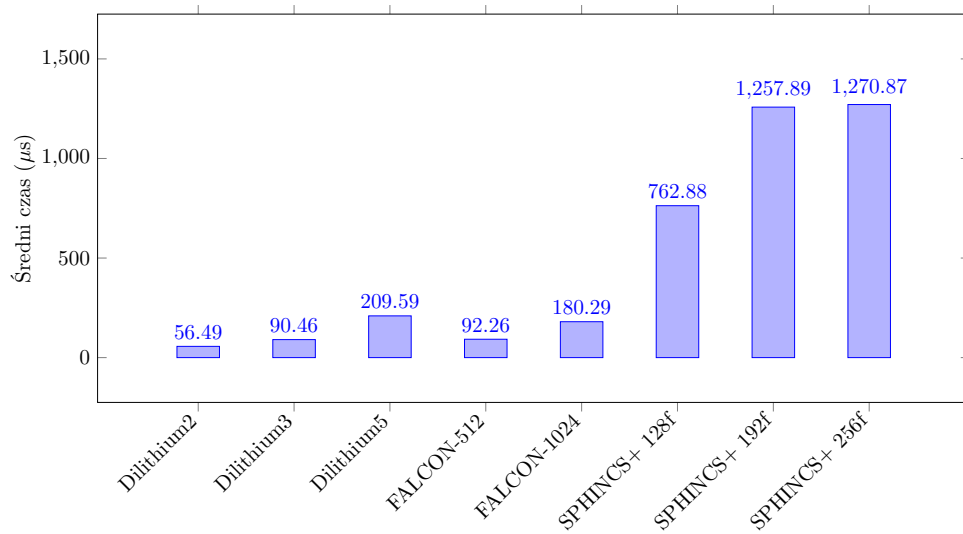
Na Rysunku 4.2 przedstawiono średni czas wykonania kroku generowania podpisu. W tym przypadku, podobnie jak dla generowania kluczy, algorytm Dilithium wypada najlepiej, ale warto zauważyć, że algorytm FALCON ma również wysoką wydajność. SPHINCS+ wymaga znacznie więcej czasu, co jest związane z jego specyficzną konstrukcją opartą na funkcjach skrótu.



RYSUNEK 4.2. Średni czas wykonania kroku generowania podpisu dla algorytmów Dilithium, FALCON oraz SPHINCS+ [24]

Na Rysunku 4.3 przedstawiono średni czas wykonania kroku potwierdzania podpisu. Tutaj również widoczna jest przewaga pod względem wydajności algorytmów opartych na kratkach, w

szczegółności Dilithium.



RYSUNEK 4.3. Średni czas wykonania kroku potwierdzania podpisu dla algorytmów Dilithium, FALCON oraz SPHINCS+ [24]

Podsumowując, wybór odpowiedniego algorytmu zależy od konkretnych wymagań aplikacyjnych. Algorytm Dilithium jest najlepszym wyborem jeśli chodzi o szybkość generowania kluczy, natomiast FALCON oferuje najlepszą wydajność pod względem generowania i weryfikacji podpisów. SPHINCS+ wyróżnia się niewielkim rozmiarem kluczy publicznych, co jest korzystne z punktu widzenia pamięci. Każdy z tych algorytmów ma swoje mocne strony, które mogą być decydujące w zależności od konkretnego scenariusza zastosowania.

TABELA 4.1. Porównanie rozmiaru klucza publicznego i podpisu algorytmów Dilithium, FALCON, SPHINCS+, RSA i ECC [25]

Algorytm	Wersja algorytmu	Poziom bezpieczeństwa NIST	Rozmiar klucza publicznego (bajty)	Rozmiar klucza prywatnego (bajty)	Rozmiar podpisu (bajty)
Sphincs+	128s	1	32	64	17,088
	192s	3	48	96	35,664
	256s	5	64	128	49,856
Dilithium	2	1	1,312	2,528	2,420
	3	3	1,952	4,000	3,293
	5	5	2,592	4,864	4,595
FALCON	512	1	897	1,281	690
	1024	5	1,793	2,305	1,330
RSA	-	-	256	256	256
ECC	-	-	64	32	256

Algorytm Dilithium, oparty na kratkach, oferuje najlepsze czasy generowania kluczy, podpisywania oraz weryfikacji podpisów, co czyni go idealnym wyborem dla aplikacji wymagających wysokiej wydajności. Jego wada to stosunkowo duży rozmiar kluczy publicznych i podpisów, co przedstawione zostało w Tabeli 4.1, która może być istotnym czynnikiem w aplikacjach o ograniczonej pamięci.

FALCON również oparty na kratkach, charakteryzuje się nieco dłuższymi czasami generowania kluczy w porównaniu do Dilithium, ale oferuje kompaktowe klucze publiczne i podpisy, co jest zaletą w przypadku aplikacji, gdzie rozmiar danych jest kluczowy. Jego wydajność w zakresie generowania i weryfikacji podpisów jest również bardzo dobra.

SPHINCS+, oparty na funkcjach skrótu, wyróżnia się najmniejszymi kluczami publicznymi, co może być decydującym czynnikiem w niektórych scenariuszach zastosowania. Jego czas generowania i weryfikacji podpisów jest dłuższy, co wynika z jego unikalnej konstrukcji. SPHINCS+ jest szczególnie atrakcyjny w kontekście aplikacji wymagających długotrwałej ochrony danych, ze względu na jego bezpieczeństwo oparte na różnych podejściach kryptograficznych.

Wybór algorytmu zależy od specyficznych wymagań aplikacji. Dla aplikacji wymagających szybkiego przetwarzania, Dilithium jest najlepszym wyborem. Jeśli kluczowe jest zmniejszenie rozmiaru kluczy, FALCON jest bardziej odpowiedni. W przypadkach, gdzie istotny jest mini-

malny rozmiar klucza publicznego i wysoki poziom bezpieczeństwa, SPHINCS+ może być preferowany.

Rozdział 5

Wnioski i perspektywy rozwoju kryptografii postkwantowej

Kryptografia postkwantowa to dziedzina, która zyskuje coraz większe znaczenie w obliczu szybkiego rozwoju technologii kwantowych. Obecne systemy szyfrowania, oparte na trudności rozwiązania matematycznych problemów, takich jak faktoryzacja dużych liczb pierwszych czy obliczenie dyskretnego logarytmu, mogą być zagrożone przez rozwój kwantowych komputerów. Kwantowe komputery, dzięki swojej zdolności do równoczesnego przetwarzania ogromnych ilości danych, mogą potencjalnie złamać istniejące protokoły szyfrowania. Wprowadzenie kryptografii postkwantowej do architektur i sieci nie dotyczy tylko nowatorskich i interesujących problemów badawczych oraz ekscytujących wyzwań inżynierskich, ale przede wszystkim ochrony komunikacji i danych wrażliwych. Jest to również kluczowy element strategii bezpieczeństwa, który kładzie nacisk na możliwość zaszyfrowania ogromnej ilości danych teraz i odszyfrowania ich dopiero wtedy, gdy komputery kwantowe będą wystarczająco rozwinięte. Atakujący mogą wykorzystać techniki i algorytmy, o których obecnie możemy nie wiedzieć, dlatego ochrona przed nimi staje się niezbędna.

W pracy została przeprowadzona szczegółowa analiza algorytmu postkwantowego podpisu cyfrowego *CRYSTALS-Dilithium*. Rozdziały pracy obejmowały omówienie podatności tradycyjnych algorytmów, takich jak RSA, na ataki kwantowe, co podkreśliło konieczność rozwijania nowych rozwiązań kryptograficznych. Dokonano analizy algorytmu Shora, który jest bezpośrednim zagrożeniem dla obecnych systemów kryptograficznych opartych na faktoryzacji liczb. Przedstawiono również proces i próby standaryzacji algorytmów kryptografii postkwantowej przez NIST, koncentrując się na wyborze algorytmów zapewniających najwyższy poziom bezpieczeństwa i wydajności.

Omówiono kryptografię opartą na kratkach, w tym kluczowe problemy i schematy, które są fundamentem dla tego podejścia, oraz jej zastosowanie w praktyce kryptograficznej. Przeprowadzono również przegląd literatury, który pozwolił na zrozumienie obecnego stanu wiedzy na temat kryptografii postkwantowej i algorytmów opartych na kratkach.

W ramach analizy algorytmu *CRYSTALS-Dilithium*, przedstawiono uproszczone kroki, rozszerzoną interpretację oraz algorytmy pomocnicze. Dodatkowo, sporządzono przykładowy przebieg algorytmu, w tym algorytmu generowania kluczy, podpisu i weryfikacji podpisu, co ułatwiło zrozumienie poszczególnych kroków i przebiegu algorytmu.

Cel pracy został osiągnięty poprzez szczegółowe zapoznanie się z algorytmem *CRYSTALS-Dilithium* i kryptografią postkwantową, zwłaszcza tą opartą na kratkach. Praca umożliwiła również zrozumienie ogólnego stanu rozwoju mechanizmów odpornych na przyszłe ataki z wykorzystaniem komputerów kwantowych oraz obecny stan przygotowania nie tylko pod względem gotowych narzędzi, ale także postępu wdrażania tych narzędzi w istniejące systemy.

Obecny stan zaangażowania dużych firm technologicznych w obszarze komputerów kwantowych jest niezwykle dynamiczny i obiecujący. Firmy takie jak Google, Microsoft, Amazon, IBM i Intel intensywnie inwestują w rozwój sprzętu kwantowego oraz technologii postkwantowych, widząc w nich potencjał do rewolucji w dziedzinie obliczeniowej, optymalizacji i sztucznej inteligencji [26]. Ich zaangażowanie odzwierciedla strategiczną potrzebę adaptacji i zdobycia przewagi konkurencyjnej w zmieniającym się krajobrazie technologicznym.

Kryptografia postkwantowa jest kluczowym elementem tych działań, ponieważ rozwój kwantowych komputerów stawia pod znakiem zapytania dotychczasowe metody szyfrowania oparte na problemach trudnych obliczeniowo, takich jak RSA. Potencjalna przewaga komputerów kwantowych w rozwiązywaniu problemów matematycznych może zagrozić bezpieczeństwu danych przesyłanych i przechowywanych w obecnym cyfrowym środowisku. Dlatego firmy te inwestują również w rozwój kryptografii postkwantowej, poszukując algorytmów odpornych na potencjalne ataki kwantowe.

Przyszłość kryptografii postkwantowej leży w opracowaniu nowych metod szyfrowania, które będą w stanie zapewnić bezpieczeństwo danych również w erze kwantowych komputerów. Dotychczasowe prace i inwestycje w tym obszarze mają na celu nie tylko zrozumienie i zabezpieczenie obecnych systemów, ale również przygotowanie podstaw do nowych rozwiązań, które będą odporniejsze na przyszłe zagrożenia. Kryptografia postkwantowa staje się więc nie tylko dziedziną badawczą, ale również kluczowym elementem strategii bezpieczeństwa cyfrowego, zarówno dla firm technologicznych, jak i dla szerokiego spektrum sektorów gospodarki, które muszą przygotować się na nadchodzącą erę kwantowych technologii.

Literatura

- [1] Lily Chen, Dustin Moody, Andrew Regenscheid, and Angela Robinson. Digital Signature Standard (DSS). Technical Report NIST FIPS 186-5, National Institute of Standards and Technology (U.S.), Gaithersburg, MD, February 2023.
- [2] Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner, and Daniel Smith-Tone. Report on Post-Quantum Cryptography. Technical Report NIST IR 8105, National Institute of Standards and Technology, April 2016.
- [3] Kelsey Jackson, Carl Miller, and Daochen Wang. *Evaluating the Security of CRYSTALS-Dilithium in the Quantum Random Oracle Model*, pages 418–446. 04 2024.
- [4] Manohar Raavi, Simeon Wuthier, Pranav Chandramouli, Yaroslav Balytskyi, Xiaobo Zhou, and Sang-Yoon Chang. Security comparisons and performance analyses of post-quantum signature algorithms. 01 2021.
- [5] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. Cryptology ePrint Archive, Paper 2020/292, 2020. <https://eprint.iacr.org/2020/292>.
- [6] Yong Liu, Yuejun Liu, Yongbin Zhou, Yiwen Gao, Zehua Qiao, and Huaxin Wang. A novel power analysis attack against CRYSTALS-dilithium implementation. Cryptology ePrint Archive, Paper 2024/111, 2024. <https://eprint.iacr.org/2024/111>.
- [7] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. High-performance hardware implementation of CRYSTALS-dilithium. Cryptology ePrint Archive, Paper 2021/1451, 2021. <https://eprint.iacr.org/2021/1451>.
- [8] Information Technology Laboratory Computer Security Division. Public-Key Post-Quantum Cryptographic Algorithms: Nominations | CSRC, December 2016. <https://csrc.nist.gov/news/2016/public-key-post-quantum-cryptographic-algorithms>.
- [9] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, February 1978.
- [10] P.W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, November 1994.

- [11] Vasileios Mavroeidis, Kamer Vishi, Mateusz D., and Audun Jøsang. The Impact of Quantum Computing on Present Cryptography. *International Journal of Advanced Computer Science and Applications*, 9(3), 2018.
- [12] Technology | IBM Quantum Computing - Our roadmap to advance useful quantum computing. <https://www.ibm.com/quantum/technology>.
- [13] Information Technology Laboratory Computer Security Division. Round 1 Submissions - Post-Quantum Cryptography | CSRC | CSRC, January 2017. <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions>.
- [14] Information Technology Laboratory Computer Security Division. PQC Standardization Process: Second Round Candidate Announcement | CSRC, January 2019. <https://csrc.nist.gov/news/2019/pqc-standardization-process-2nd-round-candidates>.
- [15] Information Technology Laboratory Computer Security Division. Three Draft FIPS for Post-Quantum Cryptography | CSRC, August 2023. <https://csrc.nist.gov/News/2023/three-draft-fips-for-post-quantum-cryptography>.
- [16] Information Technology Laboratory Computer Security Division. Announcing PQC Candidates to be Standardized, Plus Fourth Round Candidates | CSRC, March 2022. <https://csrc.nist.gov/news/2022/pqc-candidates-to-be-standardized-and-round-4>.
- [17] Nicklas Körtge. The Idea behind Lattice-Based Cryptography, June 2021. <https://medium.com/nerd-for-tech/the-idea-behind-lattice-based-cryptography-5e623fa2532b>.
- [18] Joseph Surin and Shaanan Cohnen. A gentle tutorial for lattice-based cryptanalysis. *Cryptology ePrint Archive*, Paper 2023/032, 2023. <https://eprint.iacr.org/2023/032>.
- [19] Peter Schwabe. CRYSTALS – Kyber and Dilithium, February 2018. <https://cryptojedi.org/peter/data/oxford-20180207.pdf>.
- [20] Chris Peikert. A decade of lattice cryptography. *Found. Trends Theor. Comput. Sci.*, 10(4):283–424, mar 2016.
- [21] Vadim Lyubashevsky. Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures. In *Advances in Cryptology – ASIACRYPT 2009*, volume 5912, pages 598–616. Springer Berlin Heidelberg, 2009. Series Title: Lecture Notes in Computer Science.
- [22] Shi Bai and Steven D. Galbraith. An improved compression technique for signatures based on learning with errors, 2013.
- [23] Shi Bai, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium algorithm specifications and supporting documentation (version 3.1). February 2021.
- [24] Pqc digital signature speed tests. https://asecuritysite.com/pqc/pqc_sig.
- [25] Marin Vidaković and Kruno Miličević. Performance and Applicability of Post-Quantum Digital Signature Algorithms in Resource-Constrained Environments. *Algorithms*, 16(11):518, November 2023. Number: 11 Publisher: Multidisciplinary Digital Publishing Institute.

- [26] CBINSIGHTS. The Big Tech In Quantum Report: How Google, Microsoft, Amazon, IBM, & Intel Are Battling For The Future Of Computing, May 2022.
[https://bleu-azur-consulting.eu/wp-content/uploads/2022/05/
click-here-to-access-cb-insights-big-tech-in-quantum-computing-report.pdf](https://bleu-azur-consulting.eu/wp-content/uploads/2022/05/click-here-to-access-cb-insights-big-tech-in-quantum-computing-report.pdf).



© 2024 Hubert Mamys

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX na platformie Overleaf.